

Secret-Key Sampling Watermarks for GENERator and GENERator-v2

Technical Research, Implementation, and Evaluation Manual

Research plan

12 August 2026

Contents

Executive summary	8
1. Project definition	10
1.1 What we are trying to build	10
1.2 Why use a secret key?	11
1.3 Desired properties	12
Mathematical distribution property	12
Statistical detection property	12
Cryptographic property	12
Biological-proxy property	12
1.4 What is out of scope for the first implementation	12
2. GENERator and GENERator-v2: source-grounded background	12
2.1 Original GENERator	12
2.2 Current GENERator-v2 tokenizer	13
2.3 Six-base alignment requirement	13
2.4 The current GENERator-v2 generation path	14
2.5 Two baselines that must never be conflated	14
Baseline B_{tok} : direct 6-mer token sampling	14
Baseline B_{bp} : released base-marginal generation	15
Why both are needed	15
2.6 Freeze the implementation before experiments	15
3. Why a 6-mer vocabulary may help - and why it may not	16
3.1 Nominal vocabulary is not effective vocabulary	16
3.2 A useful prediction for random keyed partitions	16
3.3 Greedy decoding is a zero-randomness boundary	17
3.4 Information per token is not information per base	17
4. Threat model and information flow	18
4.1 Parties	18
Generator/provider	18
Verifier	18
User/attacker	18
4.2 Who knows what?	18
4.3 Attacker levels	19
A0 - no malicious attacker	19
A1 - watermark existence known, details unknown	19
A2 - algorithm public, key secret	19

A3 - detector-query access	19
A4 - many watermarked sequences	19
A5 - model and generation configuration known	19
A6 - key compromise	19
5. Watermark constructions to evaluate	19
5.1 Construction P: keyed binary partition + exact coupling	19
Step 1 - secret token partition	20
Step 2 - compute probability mass of group 1	20
Step 3 - obtain the next latent bit	20
Step 4 - couple latent bit to emitted group	20
Step 5 - choose a token inside the selected group	21
Information diagnostic	21
5.2 Construction ITS: keyed inverse-transform sampling	21
5.3 Construction EXP: exponential-minimum sampling	21
5.4 DiPmark-style distribution-preserving reweighting	22
5.5 Private-key protein watermark baseline	22
5.6 PRCs are a later layer, not the first sampler	22
6. Continuous running example: one complete token-generation path	22
7. Continuous running example: eight tokens, intrinsic channel errors, and final DNA	24
7.1 The three different kinds of randomness	25
Model uncertainty	25
Watermark or code randomness	25
Final sampling randomness	25
7.2 Common misunderstanding: exact marginals do not mean zero channel errors	26
8. Standalone verification: exact mechanics	26
8.1 Detector pipeline	26
8.2 Phase enumeration for a clean fragment	27
8.3 Forward and reverse-complement orientation	27
8.4 Construction-P observation function	28
8.5 Detector result object	28
8.6 Toy clean verification	28
9. Nucleotide edits and their token-level consequences	29
9.1 Four distinct error sources	29
9.2 Substitutions	29
9.3 One insertion: why phase collapse matters	30
9.4 One deletion	30
9.5 Cropping: a bounded six-phase problem	31
9.6 Reverse complement plus crop	31
9.7 Circular sequences	31
9.8 Common misunderstanding: a PRC edit guarantee is not yet a DNA edit guarantee	31
10. Synchronization strategies to compare	31
10.1 S0 - one global phase	32
10.2 S1 - independent local blocks/windows	32
10.3 S2 - local sequence-derived watermark state	32
10.4 S3 - phase-aware edit alignment	33
10.5 S4 - explicit synchronization/edit codes	33
10.6 Experiment that decides among S0-S4	33
11. Separate the sampling channel from error correction and cryptography	34
11.1 C0 - independent uniform latent randomness	34
11.2 C1 - practical research detector / conventional error correction	34

11.3 C2 - actual PRC integration	34
11.4 Suggested common interface	35
11.5 What the code layer receives	35
12. Implementation specification	35
12.1 Baseline distribution adapter	35
12.2 Tokenizer and alignment utilities	36
12.3 Key derivation	37
12.4 Construction-P partition function	37
12.5 Exact binary coupling	37
12.6 Token selection inside a partition	38
12.7 Complete Construction-P sampler	38
12.8 ITS and EXP adapters	39
12.9 Observation function	39
12.10 Standalone sequence detector	39
12.11 Baseline-vs-watermark integration seam	39
13. Edge cases and protocol decisions	40
14. Repository layout and reproducibility contract	41
14.1 Reproducibility manifest	43
14.2 Freeze external code paths	43
15. Stable experiment IDs and project dependency graph	44
15.1 Dependency graph	44
16. Data schemas	45
16.1 generator_channel.parquet	45
16.2 generation_results.parquet	46
16.3 detection_results.parquet	47
16.4 edit_results.parquet	47
16.5 null_results.parquet	48
17. E0 - synthetic sampler validation	48
17.1 Question	48
17.2 Synthetic distributions	48
17.3 Monte Carlo scale	49
17.4 Measurements	49
17.5 Exact-marginal regression test	49
17.6 Edge-case tests for Construction P	49
$q = 0.5$	49
$q = 0.9$	50
$q = 1$	50
$q = 0$	50
17.7 Continue condition	50
18. E1 - GENERator integration and exact baseline audit	50
18.1 Question	50
18.2 Tasks	50
18.3 Reference trace	51
18.4 Important processor-order test	51
18.5 Baseline equivalence test	51
18.6 Deliverables	51
18.7 Continue condition	51
19. E2 - measure the real 6-mer sampling channel	51
19.1 Question	52

19.2 Model selection	52
19.3 Prompt/context selection	52
19.4 Decoding configurations	52
19.5 Target scale	52
19.6 Per-position metrics	53
Shannon entropy	53
Top-1 probability	53
Effective vocabulary	53
Keyed partition balance	53
Construction-P intrinsic mismatch	53
Information per token	53
Information per base	53
19.7 Random-key versus oracle upper bound	54
Detector-compatible random keyed partition	54
Oracle probability-balanced partition	54
19.8 Main plots	54
19.9 Interpretation examples	54
Example A	54
Example B	55
Example C	55
19.10 Feasibility gate	55
20. E3 - distribution-preserving generation on GENERator	55
20.1 Question	55
20.2 Fixed-state test using real GENERator distributions	56
20.3 Autoregressive matched-cohort test	56
20.4 Distribution metrics	56
20.5 Unkeyed preliminary distinguisher	56
20.6 Runtime overhead	57
20.7 Continue condition	57
21. E4 - clean standalone detection	57
21.1 Question	57
21.2 Detector variants	58
21.3 Sequence lengths	58
21.4 Nulls	58
21.5 Operating point	58
21.6 Report	58
21.7 Internal continue threshold	59
21.8 Important outcome: generic sampler already wins	59
22. E5 - nucleotide substitution experiment	59
22.1 Question	59
22.2 Rates	59
22.3 Measurements	59
22.4 Project gate	60
23. E6 - nucleotide insertions and deletions	60
23.1 Question	60
23.2 Separate insertion and deletion tests	60
23.3 Mixed edits	60
23.4 Metrics beyond TPR	60
23.5 S0 baseline expectation	61
23.6 Internal target after synchronization work	61
24. E7 - cropping, phase, orientation, and local fragments	61
24.1 Question	61

24.2 Cropping fractions	61
24.3 Phase tests	61
24.4 Orientation	61
24.5 Success condition	62
25. E8 - resynchronization comparison	62
25.1 Question	62
25.2 Compare	62
25.3 Matched experiment	62
25.4 Primary plots	62
25.5 Block-size sweep	62
25.6 Decision	62
26. E9 - null distributions, wrong keys, and system-level false positives	63
26.1 Basic definitions	63
26.2 Multiple search hypotheses	63
26.3 Preferred calibration strategy	63
26.4 Null families	64
26.5 Wrong-key sweeps	64
26.6 Empirical limits	64
26.7 Later low-FPR path	64
27. E10 - unkeyed distinguishability	65
27.1 Question	65
27.2 Data split	65
27.3 Classifiers	65
27.4 Report	65
28. E11 - detector-query removal	65
28.1 Attacker knowledge	66
28.2 Safe objective	66
28.3 Measurements	66
28.4 Redesign signal	66
29. E12 - baseline comparison	66
29.1 Required methods where implementation is feasible	67
29.2 Comparison table	67
29.3 Acceptable outcome: use generic method directly	67
30. E13 - ECC and PRC integration	68
30.1 Question	68
30.2 Conventional ECC first	68
30.3 PRC literature table	68
30.4 Composition target	68
31. One-week feasibility sprint	69
Day 1 - E1 integration and tokenizer audit	69
Question	69
Code	69
Work	69
Outputs	69
Stop/redesign	70
Day 2 - E0 completion + E2 channel measurement	70
Question	70
Code	70
Work	70
Outputs	70

Decision	70
Day 3 - E3 exact sampler on real generation	70
Question	70
Work	70
Outputs	71
Continue	71
Day 4 - E4 clean standalone detector	71
Question	71
Code	71
Work	71
Output	71
Continue	71
Day 5 - E5/E6/E7 edit characterization	71
Question	71
Work	71
Output	72
Interpretation	72
Day 6 - E8 local resynchronization	72
Question	72
Implement/compare	72
Output	72
Continue	72
Day 7 - E9/E10/E12 and decision	72
Work	72
Master table	73
Final one-week decision	73
32. Full experiment matrix after the feasibility gate	73
32.1 Models	73
32.2 Baselines	73
32.3 Sampling configurations	73
32.4 Lengths	73
32.5 Watermark schemes	74
32.6 Synchronization	74
32.7 Edits	74
32.8 Primary metrics	74
Watermark/detection	74
Channel	74
Distribution	74
Biology proxies	75
33. Biological and sequence-quality evaluation	75
33.1 Matched comparisons	75
33.2 Independent scoring	75
33.3 Safe benchmark scope	75
34. Security evaluation by attacker class	76
34.1 A0 - ordinary edits/no malicious attacker	76
34.2 A1 - watermark existence known, algorithm details unknown	76
34.3 A2 - algorithm public, key secret	76
34.4 A3 - detector-query access	76
34.5 A4 - many watermarked samples	77
34.6 A5 - model and decoding policy known	77
34.7 A6 - key compromise	77
35. Proof-obligation ledger	77
35.1 Plain-English expansion of the most important obligations	79

O1 - What is the baseline?	79
O2 - Is exact preservation actually true?	79
O3 - Does GENERator leave enough randomness?	79
O4 - Can we verify without GENERator?	79
O8 - Can indels resynchronize?	79
O9 - Is the reported FPR a system FPR?	80
O15 - Is the cryptographic composition proof real?	80
35.2 Claim-type labels	80
Mathematical	80
Cryptographic	80
Statistical/empirical	80
Biological proxy	80
36. Research thresholds and why they exist	80
37. Twelve-week execution plan	81
Weeks 1-2 - freeze generator and samplers	81
Engineering	81
Theory	81
Deliverables	81
Decision	82
Weeks 3-4 - large channel benchmark	82
Targets	82
Deliverables	82
Decision	82
Weeks 5-6 - standalone detector and finite-length behavior	82
Engineering	82
Experiments	82
Deliverables	82
Weeks 7-8 - nucleotide edits and synchronization	83
Experiments	83
Deliverables	83
Decision	83
Week 9 - nulls and security evaluation	83
Week 10 - baseline replication	83
Week 11 - ECC/PRC and biological/system checks	84
Coding	84
Quality/system	84
Week 12 - decision and paper/technical report skeleton	84
Result A - direct keyed sampler works	84
Result B - generic text sampler works, custom partition unnecessary	84
Result C - clean watermark works but indels defeat standalone verification	84
Result D - direct token distribution is too concentrated	84
38. Compute plan	84
38.1 Benchmark first, do not guess throughput	84
38.2 GPU-expensive tasks	85
38.3 CPU-cheap tasks	85
38.4 Storage planning	85
38.5 Confirmation-model strategy	85
39. Main risks and stopping conditions	85
39.1 The real token distribution may still be low entropy	85
39.2 Direct token sampling may be an artificial deployment baseline	86
39.3 Generic watermarking may already solve the clean token problem	86
39.4 Nucleotide indels may make standalone 6-mer verification impractical	86
39.5 False positives may explode under scanning	86

39.6 Key reuse may create repeated patterns	86
39.7 A score-query attacker may erase evidence cheaply	86
39.8 PRC theory may not be practical at finite genomic lengths	86
40. Decision framework	86
CONTINUE	86
CONTINUE BUT REDESIGN SYNCHRONIZATION	87
USE GENERIC WATERMARK DIRECTLY	87
STOP	87
41. Immediate implementation order	87
42. Concise technical verdict	88
43. References	88
Appendix A. Construction-P marginal derivation	89
Appendix B. Random keyed partition balance heuristic	90
Appendix C. Detector pseudocode	91
Appendix D. Experiment completion checklist	92
Before E0	92
Before E1	92
Before E2	92
Before E3	93
Before E4	93
Before indel claims	93
Before cryptographic claims	93
Before final report/paper	93

Executive summary

This document specifies a research program for adding a **secret-key, generation-time watermark to DNA produced by GENERator / GENERator-v2**, with the watermark inserted in the sampler rather than by retraining the genomic language model. The first target is deliberately narrow: a zero-bit provenance test that answers whether a DNA sequence contains evidence of generation by a participating watermarked generator. The desired verifier receives the suspect DNA, a secret verification key, and the public algorithm; it should not need the original prompt, the original model logits, or a replay of the generation process.

The main reason to study GENERator is its fixed 6-mer tokenization. For ordinary nucleotide-level generation the biological alphabet has only four symbols. A fixed 6-mer vocabulary contains all

$$4^6 = 4096$$

A/T/C/G 6-mers, potentially giving a keyed sampler far more choices at each autoregressive step. The current GENERator-v2 tokenizer indeed enumerates all 4096 DNA 6-mers, preceded by 32 special tokens, for a total vocabulary size of 4128 in the released 1.2B checkpoint. [R2, R3]

However, the current GENERator-v2 generation implementation introduces a critical complication. The released GENERatorForCausalLM.generate() does **not** simply sample one token from the 4096-way next-6-mer distribution. It appends a _BPLogitsProcessor that:

1. extracts the probability distribution over valid DNA 6-mer tokens;
2. marginalizes that distribution separately at each of the six nucleotide positions;

3. obtains six four-way distributions over A/T/C/G;
4. independently chooses one base at each position, by multinomial sampling when `do_sample=True` or by per-position argmax when `do_sample=False`;
5. reconstructs the resulting 6-mer and forces that token. [R3]

This means the project must distinguish two baselines:

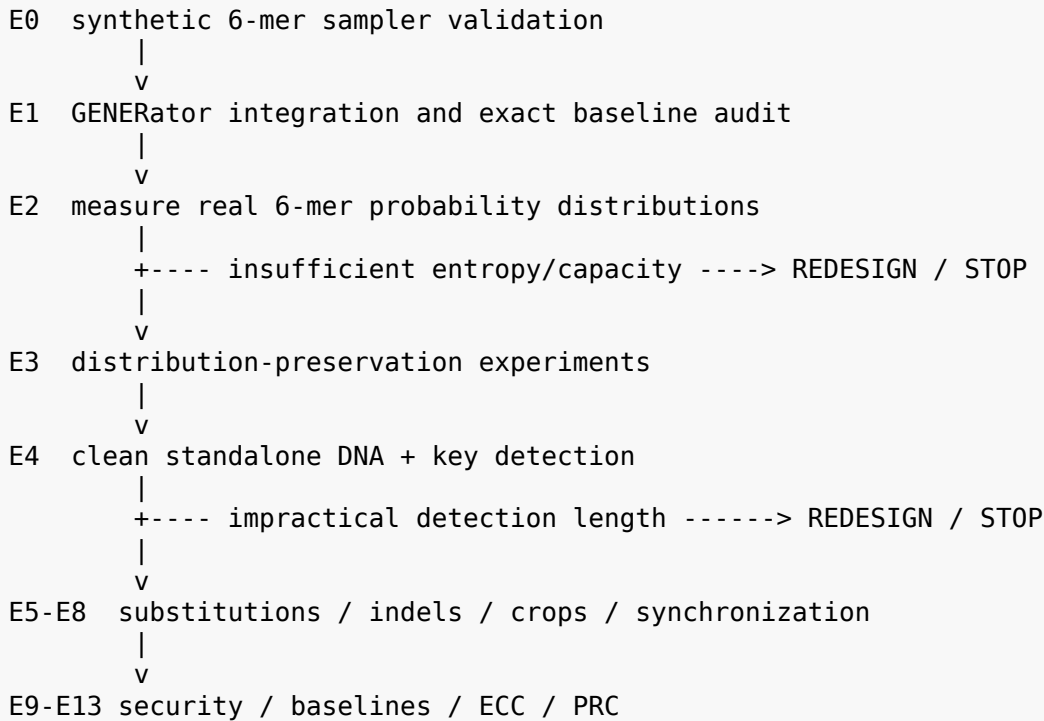
B_{tok} = direct categorical sampling from the processed 4096-way 6-mer distribution

and

B_{bp} = the released GENERator-v2 base-marginal generation procedure.

B_{tok} is the primary research baseline for testing whether a large fixed genomic vocabulary makes secret-key sampling watermarks practical. B_{bp} is a required control because it is the current released generation behavior. A watermark applied only after the current base-marginal processor would again face six four-letter sampling decisions and would not test the intended 4096-way channel.

The recommended first implementation therefore does **not** begin with a pseudorandom error-correcting code (PRC). It proceeds in layers:



Three sampler families should be implemented early:

1. **Keyed binary token partition + exact marginal coupling.** This is a simple reference construction that exposes the role of the model's effective vocabulary and is easy to test mathematically.
2. **Inverse-transform sampling (ITS)** using a secret randomized key sequence, following Kuditipudi et al. [R6].
3. **Exponential-minimum sampling (EXP)**, also following Kuditipudi et al. [R6].

The largest unresolved problem is expected to shift from **watermark capacity** to **token synchronization**. With fixed 6-mers, a clean crop creates only six possible token phases, which the detector can enumerate. A one-nucleotide insertion or deletion is harder: naïvely grouping the edited DNA into 6-mers can

change every downstream token. Therefore an edit rate stated in nucleotides cannot be directly equated to the edit rate experienced by the watermark code.

The project should be judged by measurements, not by the nominal 4096-token vocabulary. Aggressive decoding truncation or a peaked next-token distribution may reduce the effective support to a handful of candidates. The highest-value first measurement is therefore the distribution of effective vocabulary size, token entropy, keyed-partition imbalance, and expected watermark information on 100,000-1,000,000 real next-token states from GENERator.

The current recommendation is:

Proceed through E0-E4 on GENERator-v2 1.2B. Freeze and validate the direct-token baseline B_tok; implement partition coupling, ITS, and EXP; measure the real channel; require standalone clean detection before investing in indel synchronization or PRCs.

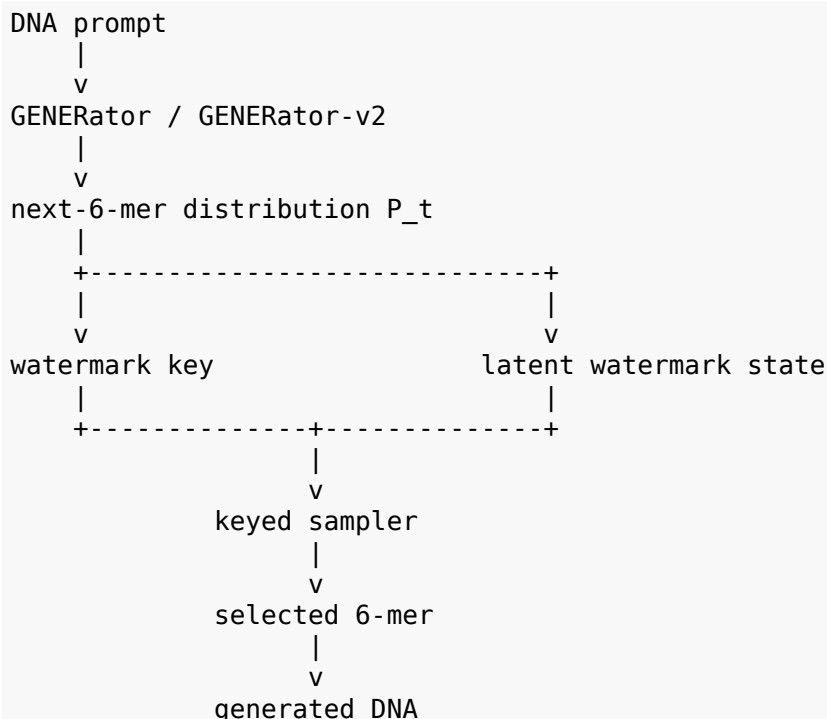
A negative result is acceptable. In particular, the study may find that 6-mer tokenization solves the capacity problem while nucleotide indels make practical standalone verification difficult. That boundary would determine whether the next effort should focus on block-local detection, local resynchronization, edit-aware alignment, or whether fixed 6-mer watermarking should be abandoned.

1. Project definition

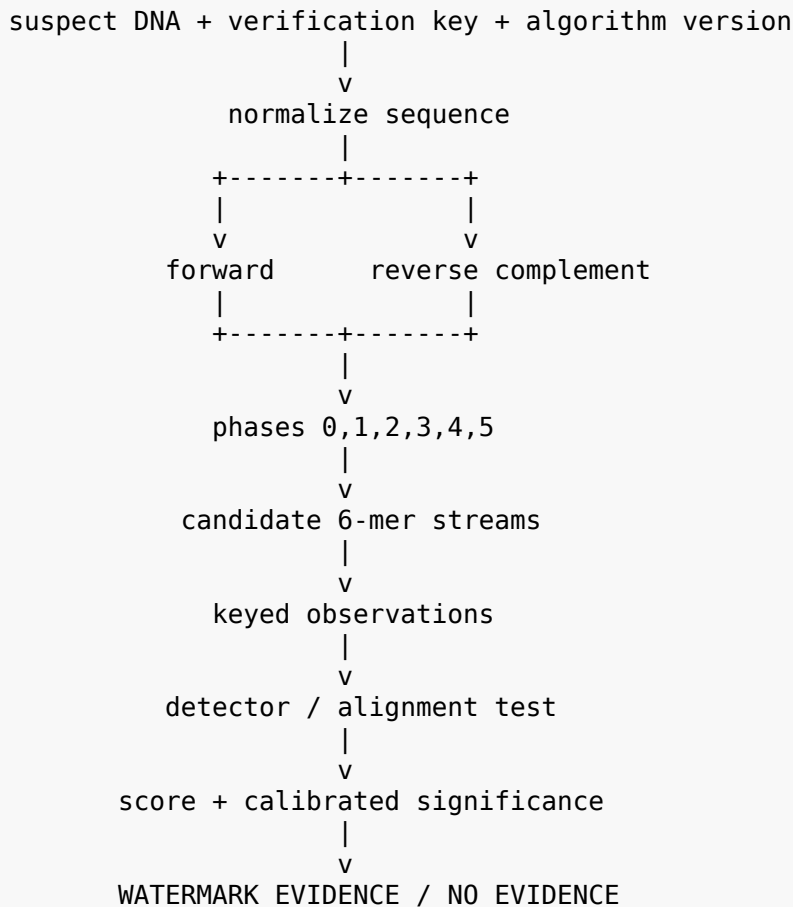
1.1 What we are trying to build

The project adds a secret-key provenance signal while GENERator produces DNA. The watermark is not a literal DNA tag and is not added after generation. It is a correlation between the sampler's random choices and secret key material.

Generation should look conceptually like this:



Verification should ideally be independent of the original model run:



The first version answers only a zero-bit question:

Does this sequence contain evidence associated with this verification key?

It should not initially attempt to embed a long payload, provide public verification, identify arbitrary providers from a global registry, or remain detectable after unlimited sequence regeneration.

1.2 Why use a secret key?

A public watermark rule gives an attacker the exact statistic that generation is trying to create. A secret key lets the algorithm itself remain public while the token-to-watermark relationship remains unknown to an attacker.

The intended security posture follows Kerckhoffs's principle: the attacker may know the complete implementation, model checkpoint, decoding settings, detector family, block length, and synchronization strategy. Security should not rely on hiding these choices.

A minimal key hierarchy is:

```

K_master
|
+-- K_sample      keyed sampling / key sequence
+-- K_partition   token partitioning, if used
+-- K_code        ECC/PRC layer, if later used
+-- K_calibration optional domain separation for tests

```

Subkeys should be derived using a standard KDF with explicit domain labels. The same raw key bytes should not be reused directly for unrelated cryptographic functions.

1.3 Desired properties

The project distinguishes four kinds of claims.

Mathematical distribution property

For a chosen deployed baseline distribution P_t , the ideal sampling claim is:

$$P_{\text{WM}}(Z_t = z \mid Z_{<t}) = P_{\text{baseline}}(Z_t = z \mid Z_{<t})$$

when marginalizing over the random secret-key material required by the construction.

This is a statement about distributions, not about one generated sequence having the same GC content.

Statistical detection property

Under the correct key, watermarked sequences should produce a detection statistic that separates them from null sequences at a specified system-level false-positive rate.

Cryptographic property

A later PRC or cryptographic construction may support a stronger statement that an efficient adversary without the relevant secret cannot distinguish watermarked from ordinary generation beyond a stated advantage under specified assumptions. This is not implied by a classifier obtaining AUC 0.5.

Biological-proxy property

Sequence-level proxies such as GC content, k-mer frequencies, model likelihood, or ORF statistics can test for gross artifacts. They do not prove biological function, viability, fitness, or safety.

1.4 What is out of scope for the first implementation

Do not build the following before the channel works:

- multi-bit provenance payloads;
- public-key verification;
- provider registries;
- wet-lab validation;
- whole-genome recombination tracking;
- claims of persistence through arbitrary evolutionary change;
- brute-force validation of 10^{-9} false-positive rates;
- elaborate synchronization strings before simpler approaches are measured;
- an implementation of the newest theoretical PRC before E2-E8 define the actual channel.

2. GENERator and GENERator-v2: source-grounded background

2.1 Original GENERator

The original GENERator is an autoregressive genomic foundation model using 6-mer tokenization. Its paper reports a 1.2B-parameter model with a 98 kb genomic context and training on 386 billion eukaryotic base pairs. The work evaluates causal genomic generation and explicitly studies tokenizer choices, reporting 6-mers as the selected generation tokenization. [R1]

For this watermark project, the important property is not benchmark rank. It is the causal interface:

$$P_t(z) = P(Z_t = z \mid Z_{<t}),$$

where a DNA token z corresponds to six nucleotides.

If $k = 6$, then the set of ordinary A/T/C/G tokens is

$$\mathcal{V}_{\text{DNA}} = \{A, T, C, G\}^6,$$

and therefore

$$|\mathcal{V}_{\text{DNA}}| = 4096.$$

2.2 Current GENERator-v2 tokenizer

The current released tokenizer constructs all 6-mers explicitly with an iterator over "ATCG" repeated k times. The tokenizer's special-token list contains 32 entries before the DNA kmers. The released 1.2B configuration reports `vocab_size = 4128`, matching 32 special tokens plus 4096 DNA 6-mers. [R2, R3]

Conceptually:

token IDs 0..31	special/control tokens
<oov> <s> </s> <pad> <mask> ... <sp2>	

token IDs 32..4127	all A/T/C/G 6-mers
AAAAAA AAAAAAT AAAAAAC AAAAAAG ... GGGGGG	

This clean separation is useful for a watermark implementation. The direct-token baseline can explicitly mask all non-DNA tokens before constructing the sampling distribution.

2.3 Six-base alignment requirement

The official model card warns that generation prompts should be aligned to a multiple of six nucleotides. If a prompt length is not divisible by six, the tokenizer may append an <oov> token, and the documentation warns that subsequent generation can become uninformative, for example repeatedly generating AAAAAA. [R2]

This means alignment is not merely a detector issue. It is part of the generator input contract.

For every generation experiment, record:

```

raw_prompt_length_bp
alignment_action
bases_truncated_or_padded
aligned_prompt_length_bp
initial_token_phase
contains_oov

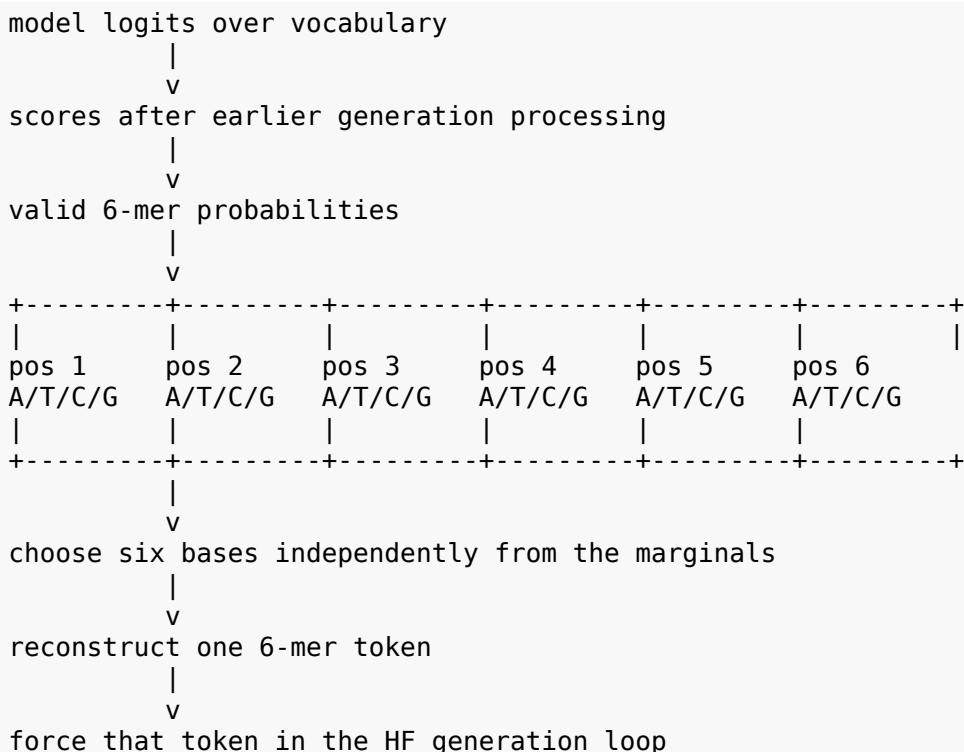
```

Do not silently trim prompts without recording it, because that changes the biological context presented to the model.

2.4 The current GENERator-v2 generation path

The current `modeling_generator.py` defines `_BPLogitsProcessor`. It begins from scores over the full vocabulary, restricts to valid k-mer token IDs, applies a softmax, and forms position-wise base marginals. For each of the six positions it sums the probabilities of all 6-mers containing A, T, C, or G at that position. It then independently chooses each position and maps the six selected bases back to a 6-mer token. [R3]

At a high level:



The source uses multinomial sampling at each base position when `do_sample=True` and per-position argmax when `do_sample=False`. It appends this processor to the logits-processor list in the model's overridden `generate()` method. [R3]

This distinction is central to the watermark project. A direct token watermark should act on a true categorical 6-mer distribution before this factorization.

2.5 Two baselines that must never be conflated

Baseline B_{tok} : direct 6-mer token sampling

Define a research baseline by taking the model's next-token scores, restricting to the 4096 DNA 6-mers, applying the chosen decoding policy, renormalizing, and drawing one 6-mer token directly.

```

raw logits
  |
  v
mask to valid DNA 6-mers
  |
  v
temperature / top-k / top-p policy
  |
  v
P_t over <=4096 6-mers
  |
  v
one categorical token draw

```

This baseline is the natural host distribution for a text-style keyed sampler.

Baseline B_{bp} : released base-marginal generation

Use the current model's ordinary `generate()` path, including `_BPLogitsProcessor`.

This is the appropriate control for claims about released GENERator-v2 behavior, but it is not a 4096-way categorical token sampler.

Why both are needed

If B_{tok} performs well as a watermark channel but B_{bp} does not, the result means that the model's learned 6-mer distribution contains useful sampling freedom but the released base-level generation policy discards or transforms that structure.

If both work, the project can compare which representation gives better detection per nucleotide and which survives edits better.

If only B_{bp} is acceptable biologically or by the model authors' intended use, then the project must be explicit that a B_{tok} watermark changes the deployed decoding procedure even if it preserves the B_{tok} distribution exactly.

2.6 Freeze the implementation before experiments

Because remote model code can change, every experiment manifest must include:

```

model_repo
model_revision
model_weight_hash or HF revision
modeling_generator.py hash
tokenizer.py hash
vocab.txt hash
transformers_version
torch_version
cuda_version
flash_attention_version
checkpoint
sampling_baseline = B_tok | B_bp
sampling_config
git_commit_of_watermark_code

```

Do not cite only main in a paper artifact.

3. Why a 6-mer vocabulary may help - and why it may not

3.1 Nominal vocabulary is not effective vocabulary

The nominal DNA vocabulary contains 4096 6-mers, but a watermark can only use probability mass the deployed sampler actually leaves available.

If top-k truncation sets

$$k = 4,$$

then the effective support has at most four tokens, regardless of the tokenizer vocabulary size.

Likewise, a highly peaked distribution such as

TTGGCA	0.97
GCTTAC	0.01
AACGTA	0.006
...	

behaves more like a small-alphabet channel than a 4096-way channel.

Therefore E2 must report at least four notions of diversity:

1. Shannon entropy;
2. top-1 probability;
3. nonzero support after decoding truncation;
4. an effective vocabulary size derived from concentration.

A useful concentration statistic is the inverse Simpson quantity

$$N_{\text{eff}} = \frac{1}{\sum_z P_t(z)^2}.$$

It equals 4096 for a perfectly uniform 4096-token distribution, but falls toward 1 as one token dominates.

3.2 A useful prediction for random keyed partitions

Suppose each token is independently assigned to hidden group 1 with probability $1/2$ under a keyed pseudorandom partition. Let

$$q_t = \sum_z P_t(z) B_z,$$

where $B_z \in \{0, 1\}$ is the secret group assignment.

Then, over random partitions,

$$\mathbb{E}[q_t] = \frac{1}{2}$$

and

$$\text{Var}(q_t) = \frac{1}{4} \sum_z P_t(z)^2 = \frac{1}{4N_{\text{eff}}}.$$

Thus the standard deviation of q_t around $1/2$ is

$$\sigma_q = \frac{1}{2\sqrt{N_{\text{eff}}}}.$$

When many tokens contribute and a normal approximation is reasonable,

$$\mathbb{E}[|q_t - 1/2|] \approx \frac{1}{\sqrt{2\pi N_{\text{eff}}}}.$$

This gives an immediate intuition for the partition-coupling channel:

N_{eff}	approximate $E q - 0.5 $
4096	0.0062
1024	0.0125
256	0.0249
64	0.0499
16	0.0997
4	0.1995

These are not guarantees for GENERator. They are planning values that explain why **effective** vocabulary is the statistic that matters.

3.3 Greedy decoding is a zero-randomness boundary

For exact greedy decoding,

$$Z_t = \arg \max_z P_t(z).$$

If the watermarked generator is required to preserve this deterministic output exactly, then there is no sampling freedom at that position. A secret sampler cannot encode information while returning the same token for every key.

Therefore greedy decoding should be included as a boundary control rather than described as a watermark setting.

The practical frontier to measure is:

```

greedy -> top-k 2 -> top-k 4 -> top-k 8 -> top-k 16 -> broader stochastic sampling
|           |           |           |           |
+-----+-----+-----+-----+-----+
                        measure
                    entropy / effective vocabulary / detection power

```

3.4 Information per token is not information per base

A 6-mer token emits six nucleotides. Every result should therefore report both token-normalized and base-normalized rates.

If a channel carries

$$I_{\text{token}} = 0.9 \text{ bits/token},$$

then the corresponding rate is

$$I_{bp} = \frac{0.9}{6} = 0.15 \text{ bits/bp.}$$

This normalization is required for fair comparison with a nucleotide-level model.

4. Threat model and information flow

4.1 Parties

Generator/provider

Runs GENERator, knows the generation secret, sees model probabilities, and produces watermarked DNA.

Verifier

Has a verification key and suspect sequence. Depending on deployment, it may be the same organization as the provider or a trusted external verifier.

User/attacker

Receives generated DNA and may crop, mutate, reverse-complement, locally replace, or otherwise edit it. The default security assumption is that the algorithm is public and only the key remains secret.

4.2 Who knows what?

Information	Generator	Verifier	A2 attacker
public	yes	yes	yes
watermark			
algorithm			
exact model	yes	not required	may know
family			
model weights	yes	not required	may know
prompt	yes	not required	may know
current log-its/probabilities	yes	no	no for past run
decoding	yes	ideally not required	may know
configuration			
generated DNA	yes	yes	yes
secret master	yes	trusted verifier or derived	no
key		key	
per-generation	yes	encoded/derivable or	may be public
nonce/offset		searched	
keyed token	computed	reconstructed/tested from	unknown
relation		key	
original	yes or reproducible	not required	no
random draws			

The most consequential row is the probability vector. Any watermark construction requiring the verifier to know the original P_t silently becomes a model-replay detector unless the necessary state is carried in the output.

4.3 Attacker levels

A0 - no malicious attacker

Sequencing-like substitutions/indels, cropping, strand reversal, and benign transformations.

A1 - watermark existence known, details unknown

Weak baseline only.

A2 - algorithm public, key secret

Default security scenario. The attacker knows the tokenizer, sampler construction, block length, detector family, and thresholding policy.

A3 - detector-query access

Test both:

- binary watermark / no watermark responses;
- numerical detector scores.

Score access can reveal a gradient-like signal even when the secret key is unknown.

A4 - many watermarked sequences

Tests repeated-key leakage, reused offsets, correlations, and whether a classifier learns provider-specific artifacts.

A5 - model and generation configuration known

The attacker can use GENERator as a proposal model for local resampling but still lacks the watermark key.

A6 - key compromise

Out of scope for watermark secrecy. The system should document rotation/revocation, not pretend the watermark remains secret after key theft.

5. Watermark constructions to evaluate

The first study should not assume one keyed sampler is best. It should implement at least three families with the same B_tok baseline and evaluation harness.

5.1 Construction P: keyed binary partition + exact coupling

This construction is useful because its mechanics and failure modes are transparent.

Step 1 - secret token partition

For each candidate DNA token z , compute a pseudorandom bit:

$$g_K(z, s_t) \in \{0, 1\},$$

where s_t is optional public/reconstructable state such as block ID or canonical local context.

A simple prototype is:

```
bit = HMAC-SHA256(K_partition, state || token_id)[0] & 1
```

Do not use this exact string encoding without specifying endianness, domain labels, and canonical serialization in code.

Step 2 - compute probability mass of group 1

$$q_t = \sum_{z: g_K(z, s_t)=1} P_t(z).$$

Step 3 - obtain the next latent bit

Let

$$C_t \sim \text{Bernoulli}(1/2)$$

for E0. Later C_t can come from a keyed stream, ECC, or PRC.

Step 4 - couple latent bit to emitted group

We choose an observed group bit Y_t whose marginal is $\text{Bernoulli}(q_t)$ while maximizing agreement with C_t .

For $q_t \geq 1/2$:

$$P(Y_t = 1 \mid C_t = 1) = 1$$

and

$$P(Y_t = 1 \mid C_t = 0) = 2q_t - 1.$$

For $q_t < 1/2$:

$$P(Y_t = 1 \mid C_t = 0) = 0$$

and

$$P(Y_t = 1 \mid C_t = 1) = 2q_t.$$

The resulting intrinsic mismatch probability is

$$P(Y_t \neq C_t) = |q_t - 1/2|.$$

Step 5 - choose a token inside the selected group

Condition on $Y_t = y$ and sample proportionally:

$$P(Z_t = z \mid Y_t = y) = \frac{P_t(z)}{\sum_{z': g_K(z', s_t) = y} P_t(z')}$$

for tokens in the selected group.

With uniform latent bits, the marginal over emitted tokens reconstructs P_t exactly.

Information diagnostic

For the binary grouping channel,

$$I(C_t; Y_t) = h_2(q_t) - \frac{1}{2}h_2(|2q_t - 1|),$$

where

$$h_2(p) = -p \log_2 p - (1 - p) \log_2 (1 - p).$$

This is an E2 diagnostic, not a promise of end-to-end detector rate after coding, block search, and edits.

5.2 Construction ITS: keyed inverse-transform sampling

Kuditipudi et al. define a distortion-free decoder Γ that maps random key material and a categorical probability distribution to a token, such that marginalizing over the random key material reproduces the original distribution. [R6]

The simplest intuition is inverse-CDF sampling:

1. the secret key sequence supplies a uniform value $u_t \in [0, 1]$ and any required keyed permutation/order;
2. the model probabilities define cumulative intervals;
3. the interval containing u_t selects the token;
4. the detector uses the observed token and key sequence to score consistency without needing the original prompt/model.

Do not reimplement ITS from a prose description alone. The implementation task should port the authors' published algorithm and tests, then replace the text vocabulary with the 4096 DNA-token vocabulary.

The important evaluation question is whether a fixed 6-mer stream produces enough statistical power at practical nucleotide lengths and whether the published alignment strategy behaves sensibly when a one-nucleotide indel retokenizes the downstream DNA.

5.3 Construction EXP: exponential-minimum sampling

The same work studies an exponential-minimum decoder. In one common notation, key material supplies independent $\xi_{t,z} \sim \text{Unif}(0, 1)$ values and the sampled token is

$$Z_t = \arg \min_z \frac{-\log \xi_{t,z}}{P_t(z)}.$$

This is a standard exponential-race way to sample from the categorical distribution when the ξ values are independent. The watermark correlation comes from generating these random values deterministically from secret key material that the verifier can later test. [R6]

EXP was the more statistically powerful of the two constructions in the original text experiments, so it should be an early GENERator baseline. [R6]

5.4 DiPmark-style distribution-preserving reweighting

DiPmark is relevant because it explicitly targets distribution preservation and detection without the original LM API or prompt. It uses random vocabulary permutations/partitions and a distribution-preserving reweighting rule. [R7]

The purpose of adapting DiPmark is not novelty. It is to determine whether a generic accessible distribution-preserving text watermark already works well on a 4096-token genomic vocabulary.

5.5 Private-key protein watermark baseline

Chen et al. demonstrate private-key watermarking of autoregressively designed protein sequences. Their framework modifies a per-residue generation distribution based on a watermark code derived from a private key, while maintaining equality of the expected modified distribution to the original distribution over watermark codes. Their detector takes a sequence + private key and does not require the original model logits. [R8]

This is the closest biological-sequence precedent and should be treated as a direct conceptual baseline. The simplest experiment is:

replace the 20-amino-acid vocabulary in the published framework with the 4096 DNA-token vocabulary and measure whether it works essentially unchanged.

If it does, that is a valid engineering result even if it removes the need for a new sampler construction.

5.6 PRCs are a later layer, not the first sampler

Pseudorandom error-correcting codes add a cryptographic goal: codewords should be computationally indistinguishable from random to an efficient observer without the decoding capability, while remaining recoverable under specified error channels. [R10]

They should be considered only after the actual DNA-to-detector channel is measured. Recent edit-oriented PRC results are theoretically relevant, but their edit rates, alphabets, assumptions, and finite-length behavior cannot be translated directly into a claim such as “the DNA watermark tolerates 5% indels.” [R11, R12]

6. Continuous running example: one complete token-generation path

The following numbers are illustrative. They are chosen so a reader can follow the algorithm by hand; they are not measured GENERator probabilities.

Assume the current aligned DNA prefix ends at a 6-mer boundary:

```
... ACGTTA | GCACTG | TTAGGC
```

Suppose the deployed B_tok policy produces the following partial distribution:

6-mer	probability	secret group
TTGGCA	0.18	1
CGATGC	0.15	0
GGAACC	0.12	1
ATCGTG	0.10	0
AATTCC	0.09	1
GCTTAC	0.08	0
TCGCGA	0.07	1
CTAGGT	0.06	1
all other valid 6-mers	0.15	mixed

Assume the total secret group-1 mass over all 4096 tokens is

$$q_t = 0.53.$$

The next latent watermark bit is

$$C_t = 1.$$

Because $q_t \geq 1/2$, the maximal coupling sets

$$P(Y_t = 1 \mid C_t = 1) = 1.$$

So the selected group is definitely group 1 for this step.

Inside group 1, the sampler renormalizes the original model probabilities. Suppose the group-1 total is 0.53 and TTGGCA has probability 0.18. Its conditional probability becomes

$$P(Z_t = \text{TTGGCA} \mid Y_t = 1) = \frac{0.18}{0.53} \approx 0.340.$$

Assume the conditional draw returns TTGGCA.

The state transition is then:

```
latent bit C_t      : 1
observed group Y_t  : 1
selected token      : TTGGCA
bases appended      : T T G G C A
```

and the DNA grows from

```
... ACGTTA | GCACTG | TTAGGC
```

to

```
... ACGTTA | GCACTG | TTAGGC | TTGGCA
```

Nothing visible marks TTGGCA as a watermark token. Under the exact marginal construction, the token still has its ordinary baseline probability after averaging over uniform latent bits.

7. Continuous running example: eight tokens, intrinsic channel errors, and final DNA

A one-token example explains the sampler but not the detector. Detection is a statistical or coded decision over many observations, so this section extends the same example across eight generated 6-mers.

The numbers below remain illustrative. They are not measurements from GENERator-v2.

Assume the latent code supplies

$$C_{1:8} = (1, 0, 1, 1, 0, 0, 1, 0).$$

At every step the keyed partition is recomputed from the key and the chosen public state rule. The exact group mass q_t therefore changes because both the model distribution and, potentially, the keyed partition change.

step t	latent bit C_t	illustrative q_t	observed group Y_t	emitted 6-mer	intrinsic mismatch?
1	1	0.53	1	TTGGCA	no
2	0	0.48	0	CGATGC	no
3	1	0.57	1	GGAACC	no
4	1	0.72	0	ATCGTG	yes
5	0	0.46	0	AATTCC	no
6	0	0.51	0	GCTTAC	no
7	1	0.55	1	TCGCGA	no
8	0	0.43	0	CTAGGT	no

The detector-visible binary stream is therefore

$$Y_{1:8} = (1, 0, 1, 0, 0, 0, 1, 0),$$

while the intended latent stream was

$$C_{1:8} = (1, 0, 1, 1, 0, 0, 1, 0).$$

The fourth symbol differs **without any mutation to the DNA**.

That distinction is central to the project:

```

latent code
  |
  | sampler-induced channel noise
  v
watermark observations encoded in generated 6-mers
  |
  | later DNA edits
  v
observations available to the verifier

```

The generated 48-bp example is

```

TTGGCA | CGATGC | GGAACC | ATCGTG |
AATTCC | GCTTAC | TCGCGA | CTAGGT

```

or, without token separators,

```
TTGGCACGATGCGGAACCATCGTGAATTCCGCTTACTCGCGACTAGGT
```

The vertical bars are **not** present in the DNA. They are shown only to make the original token boundaries visible to the reader.

7.1 The three different kinds of randomness

A recurring implementation mistake is to refer to all randomness as “the random seed.” There are at least three conceptually different sources.

Model uncertainty

GENERATOR supplies a probability vector

$$P_t(z) = P(Z_t = z \mid Z_{<t}).$$

This is not secret. It is a model property conditioned on the generated prefix and decoding policy.

Watermark or code randomness

The secret-key mechanism supplies the latent state: for Construction P this includes C_t and the keyed token partition; for ITS/EXP it includes the appropriate pseudorandom key material.

This is the source whose unpredictability to an attacker matters.

Final sampling randomness

After a group or distribution-preserving transformation has been selected, a random draw may still be required to choose the actual token. This randomness must be implemented in exactly the manner required by the chosen sampler. It must not accidentally reuse deterministic random values across generations.

A useful implementation table is:

Random object	Generator needs it?	Detector needs identical value?	Secret?	Fresh across generations?
model probability vector P_t	yes	ideally no	no	determined by prefix
master key generation	yes	yes/trusted verifier derivable or carried by protocol	yes not necessarily public	long-lived possible yes
nonce/encoding randomness	yes	yes, reconstructed	yes via key	depends on design
keyed partition/state	yes	usually no	no security assumption unless construction says otherwise	yes

Do not implement all rows by calling the same seeded `torch.Generator` unless the chosen construction explicitly allows that state reuse.

7.2 Common misunderstanding: exact marginals do not mean zero channel errors

For Construction P, exact preservation means

$$P_{\text{WM}}(Z_t = z \mid Z_{<t}) = P_t(z)$$

when the latent bit has the required distribution and the construction assumptions hold.

It does **not** mean

$$Y_t = C_t$$

at every position. In fact, when the keyed group has probability $q_t \neq 1/2$, the reference maximal coupling necessarily introduces mismatch probability

$$|q_t - 1/2|.$$

The coding/detection layer must therefore tolerate noise that is present before any external DNA edit occurs.

8. Standalone verification: exact mechanics

The desired verifier takes only:

```
suspect DNA
verification key
algorithm/version identifier
public detector configuration
```

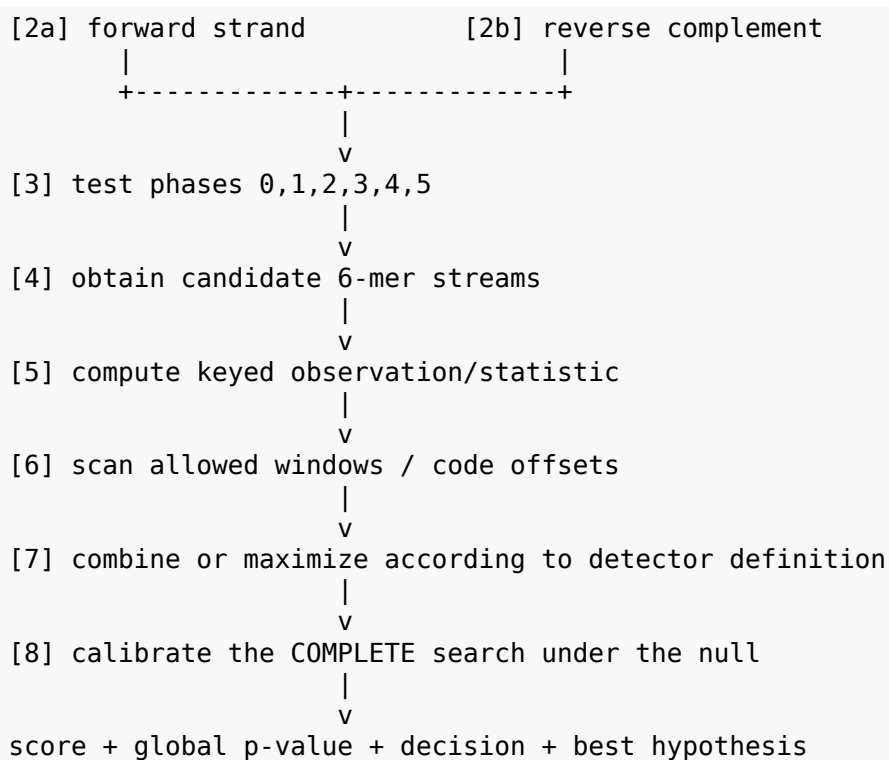
and should not require:

```
GENERATOR weights
original prompt
original model logits
original generation seed
original GPU/runtime
```

Whether this can be achieved depends on the sampler. Construction P is particularly useful as a reference because its detector-side token-to-bit mapping can be made model-independent. ITS and EXP use key alignment statistics rather than the same binary map; their detector must follow the original construction rather than being forced into the partition interface.

8.1 Detector pipeline

```
suspect DNA + key
  |
  v
[1] normalize characters / validate alphabet
  |
  +-----+
  |               |
  v               v
```



The detector must not take the smallest nominal p-value across all phases/windows and report it as though only one test had been performed. The full search is part of the statistic and must be calibrated as a whole.

8.2 Phase enumeration for a clean fragment

For a DNA string y and phase $\phi \in \{0, 1, 2, 3, 4, 5\}$, define the candidate tokenization

$$T_{\phi}(y) = (y_{\phi:\phi+6}, y_{\phi+6:\phi+12}, \dots),$$

using zero-based indexing conceptually.

For a clean crop, exactly one phase modulo six corresponds to the original token boundaries, apart from edge truncation. Therefore the uncertainty introduced by losing the original coordinate zero is bounded by six hypotheses.

8.3 Forward and reverse-complement orientation

DNA may be supplied in either strand orientation. The first detector should simply evaluate

$$D(y)$$

and

$$D(\text{revcomp}(y))$$

and calibrate the maximum score across both. With six phases, this gives twelve basic orientation/phase hypotheses before windows or code offsets are added.

Do not design a mathematically orientation-invariant code before verifying that this simple factor-of-two search is actually a bottleneck.

8.4 Construction-P observation function

For a clean aligned token z_t , a detector-compatible partition construction computes

$$\widehat{Y}_t = g_K(z_t, s_t),$$

where s_t is whatever public/reconstructable state the partition schedule requires.

The most basic stateless version is

$$g_K(z) = \text{bit}(\text{PRF}(K_{\text{partition}}, \text{domain} \parallel z)).$$

For balanced deterministic assignment over all 4096 DNA tokens, do not simply trust a hash to create exactly 2048 labels of each type if exact cardinality matters. A clean implementation can key a pseudo-random permutation of the 4096 valid token IDs and assign the first half to group 0 and the second half to group 1. Probability mass will still vary by P_t , but token counts are exactly balanced.

8.5 Detector result object

The API should return enough information for calibration and debugging:

```
@dataclass
class DetectionResult:
    score: float
    p_value: float | None
    global_p_value: float | None
    decision: bool
    best_orientation: str
    best_phase: int
    best_window_start_bp: int | None
    best_window_end_bp: int | None
    best_code_offset: int | None
    n_hypotheses: int
    n_tokens_used: int
    diagnostics: dict
```

Do not make decision the only output during research. The score distributions are needed to understand failure modes.

8.6 Toy clean verification

Using the eight-token example, suppose the verifier finds the correct orientation and phase and obtains

```
TTGGCA CGATGC GGAACC ATCGTG AATTCC GCTTAC TCGCGA CTAGGT
```

The keyed mapping yields

```
1 0 1 0 0 0 1 0
```

The expected latent pattern for the relevant code offset is

```
1 0 1 1 0 0 1 0
```

A toy Hamming score would count seven matches out of eight. A real detector should use the score associated with the chosen sampler/code rather than this toy count, but the example shows what “standalone” means mechanically: the model probabilities never appear during verification.

9. Nucleotide edits and their token-level consequences

The 6-mer vocabulary makes the clean sampling problem easier than a four-letter vocabulary may be, but it creates a new interface problem: users edit **nucleotides**, while the watermark lives in a **token stream**.

The detector must measure the induced token channel rather than assume a nucleotide edit percentage equals the same percentage of watermark-symbol errors.

9.1 Four distinct error sources

Source	DNA length changes?	Original 6-mer phase preserved?	Main detector effect
intrinsic sampler mismatch	no	yes	watermark-symbol substitution/noise
nucleotide substitution	no	yes globally	one aligned 6-mer may become another token
nucleotide insertion/deletion	yes	no after edit under naive parsing	token substitutions plus insertion/deletion/phase shift
cropping	removes prefix/suffix	phase unknown but constant	six candidate phases, fewer observations

9.2 Substitutions

A single nucleotide substitution inside a correctly aligned 6-mer changes at most that 6-mer token. The downstream phase remains correct.

Example:

```
original:
TTGGCA | CGATGC | GGAACC | ATCGTG

one substitution in token 2:
TTGGCA | CGTTGC | GGAACC | ATCGTG
          ^
```

The second token may therefore map to a different watermark observation, but tokens 3 and 4 remain aligned.

If every nucleotide is independently substituted with probability p , an aligned 6-mer is untouched only if all six bases survive:

$$P(\text{6-mer unchanged}) = (1 - p)^6.$$

Therefore

$$P(\text{6-mer contains at least one substitution}) = 1 - (1 - p)^6.$$

Planning values are:

nucleotide substitution rate p	probability an aligned 6-mer contains ≥ 1 substitution
0.1%	0.60%
0.5%	2.96%
1%	5.85%
2%	11.42%
5%	26.49%

This table is an upper bound on **token corruption**, not automatically the watermark-bit error rate: a changed 6-mer can still happen to map to the same hidden group or contribute a detector statistic consistent with the key.

9.3 One insertion: why phase collapse matters

Start from the toy sequence:

```
TTGGCA | CGATGC | GGAACC | ATCGTG |
AATTCC | GCTTAC | TCGCGA | CTAGGT
```

Insert one A inside the third token region. One illustrative resulting DNA, when naïvely reparsed from the original phase, becomes:

```
TTGGCA | CGATGC | GGAAAC | CATCGT |
GAATTC | CGCTTA | CTCGCG | ACTAGG | ...
```

Only one nucleotide was inserted, but most downstream 6-mers differ from the original token stream.

This is the key distinction:

```
one nucleotide insertion
      |
      v
+1 nucleotide offset
      |
      v
wrong modulo-6 token phase
      |
      v
many downstream token identities change
```

A detector that insists on one global phase can therefore fail catastrophically even at a tiny nucleotide indel rate.

9.4 One deletion

Deletion is symmetric at the phase level.

```
before:
TTGGCA | CGATGC | GGAACC | ATCGTG | AATTCC ...
```

```
remove one nucleotide:
TTGGCA | CGATGC | GGAAC_ | ATCGTG | AATTCC ...
```

```
naive reparsing after concatenation:
TTGGCA | CGATGC | GGAACA | TCGTGA | ATTCCG ...
```

Again, one local DNA event creates a long suffix of different token identities unless the detector resynchronizes.

9.5 Cropping: a bounded six-phase problem

Cropping does not create a changing phase. It merely hides the original coordinate origin.

Suppose a fragment begins four bases after an original token boundary. Testing all six candidate phases finds that phase 4 reconstructs:

```
GGAACC | ATCGTG | AATTCC | GCTTAC
```

for the corresponding clean region of the running example.

This is why clean fragment detection should be solved **before** insertion/deletion synchronization: it is a simpler and separately useful capability.

9.6 Reverse complement plus crop

A clean fragment with unknown strand has at most

$$2 \times 6 = 12$$

basic orientation/phase possibilities. The detector should evaluate all twelve and calibrate the best score.

9.7 Circular sequences

For a circular sequence, the biological coordinate zero is arbitrary. A local-window detector can handle this by appending enough prefix to the end to cover the largest scan window and testing windows across the wrap boundary. The null calibration must include the increased number of windows.

Do not require the original model-generation start coordinate for a forensic detector.

9.8 Common misunderstanding: a PRC edit guarantee is not yet a DNA edit guarantee

Suppose a code paper proves recovery after a fraction ρ of bit insertions/deletions/substitutions. It does **not** follow that this system tolerates a fraction ρ of nucleotide edits.

The actual chain is

DNA edits $\rightarrow$ retokenization/phase effects $\rightarrow$ watermark-observation edits $\rightarrow$ code channel.

The middle mapping must be measured or bounded first.

10. Synchronization strategies to compare

The project should not commit to an elaborate edit code until the simple methods have been measured. Use the following progression.

10.1 S0 - one global phase

Detector:

1. choose orientation;
2. choose one of six phases;
3. tokenize the entire fragment using that phase;
4. score the resulting stream.

This is the required baseline.

Expected behavior:

- clean full sequence: good;
- clean crop: good after six-phase search;
- substitutions: local damage;
- one indel: potentially permanent downstream failure.

If S0 unexpectedly survives the intended indel rates because windows often fall entirely between edits, record that result rather than assuming failure.

10.2 S1 - independent local blocks/windows

Instead of treating a whole sequence as one codeword, accumulate independently verifiable evidence in local regions.

Example for a 24-kb generated sequence and 2-kb blocks:

```
| --2 kb-- | --2 kb-- | --2 kb-- | ... | --2 kb-- |  
W1         W2         W3         W12
```

A 4-kb forensic fragment can overlap several blocks. An indel inside one block need not destroy the phase hypothesis of a distant block if the detector re-runs the six-phase search locally.

Tradeoff:

smaller blocks

- + local recovery after phase changes
- + fragment localization
- fewer watermark observations per block
- more hypotheses and larger false-positive correction

larger blocks

- + more observations for a single score
- more suffix damage after an indel

Candidate block/window sizes should be derived from clean detection length, not selected in advance. Initial sweep:

$$L \in \{120, 240, 480, 960, 1920, 3840, 7680\} \text{ bp}$$

with exact multiples of six.

10.3 S2 - local sequence-derived watermark state

A partition or key offset may be derived from a canonical local context that the verifier can recompute from DNA.

Conceptually:

$$s_t = H_K(\text{canonical}(y_{t-r:t})).$$

After a local edit, generator-side and verifier-side states can differ while the edited base remains inside the context window, then potentially match again later.

This approach must answer all of the following before implementation:

- Is context measured in nucleotides or complete 6-mers?
- Does it include the current token or only preceding DNA?
- How are the first r positions handled?
- How are ambiguous bases handled?
- Does a block boundary reset state?
- Is reverse-complement canonicalization used?
- Do inserted bases become part of the verifier context? Usually yes, because that is what the verifier observes.
- Does repeated context cause repeated key state and detectable correlations?

Treat these as protocol definitions, not incidental coding details.

10.4 S3 - phase-aware edit alignment

Instead of demanding one tokenization, use dynamic programming to align detector evidence across candidate phases and indel transitions.

A useful state can include:

(reference/key offset, observed nucleotide position, phase mod 6)

with transitions for:

- consume one aligned 6-mer;
- skip an observed nucleotide (candidate insertion);
- skip expected alignment position (candidate deletion);
- restart/localize after a block boundary.

This is conceptually related to edit-aware watermark detectors and sequence alignment. The scoring function must be tied to the chosen watermark sampler; do not substitute an arbitrary alignment score and then call the resulting p-values calibrated.

10.5 S4 - explicit synchronization/edit codes

Only if S1-S3 are inadequate, investigate:

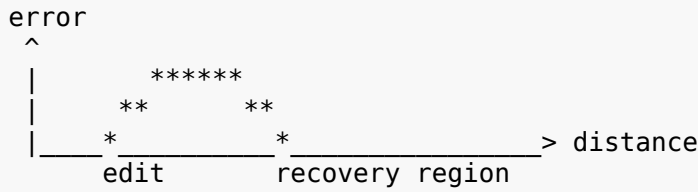
- synchronization strings [R15];
- indexing codes / edit-distance PRCs [R14];
- marker constructions;
- later binary/edit PRCs [R10-R12].

These tools can provide useful theory, but they may add substantial finite-length overhead. Their alphabet and channel assumptions must be translated into the measured 6-mer observation channel.

10.6 Experiment that decides among S0-S4

For every simulated nucleotide indel, log its original location. Plot detector/token error as a function of distance from the nearest edit.

A successful local-resynchronization method should show a curve resembling:



rather than a permanent elevated error floor for all downstream positions.

11. Separate the sampling channel from error correction and cryptography

The sampler and the code solve different problems. Keeping them behind a clean interface makes it possible to learn about the GENERator channel before committing to cryptographic machinery.

11.1 C0 - independent uniform latent randomness

For Construction P, use independent fair bits.

Purpose:

- prove/test exact marginal preservation;
- measure intrinsic channel noise;
- measure information per token/base;
- debug the GENERator integration.

There is no provenance detector security claim at this stage.

11.2 C1 - practical research detector / conventional error correction

Use a keyed pseudorandom target sequence and a conventional error-correcting/statistical detector, or use ITS/EXP directly with their corresponding key statistic.

Purpose:

- obtain clean finite-length detection curves;
- study nucleotide substitutions/crops/indels;
- compare synchronization methods;
- measure system-level false positives.

Do **not** call CSPRNG + ECC a PRC unless the actual construction satisfies the PRC definition and proof assumptions.

11.3 C2 - actual PRC integration

Only after E0-E8 establish a usable channel, select a concrete PRC or state a conditional theorem.

The theoretical goal is of the form:

If latent code distribution C_K is computationally indistinguishable from uniform and the watermark sampler with uniform latent randomness reproduces the chosen baseline generation distribution, then an efficient observer of generated DNA should not distinguish the watermarked generator from that baseline beyond the underlying code/sampler advantage, subject to the stated adaptive-generation conditions.

That statement requires a real reduction. It is not established merely by placing the words “PRC” and “distribution-preserving sampler” next to each other.

11.4 Suggested common interface

```
class LatentCode(Protocol):
    def reset(
        self,
        *,
        master_key: bytes,
        generation_nonce: bytes,
        target_tokens: int,
    ) -> None:
        ...

    def next_symbol(self) -> int:
        ...

    def detect(
        self,
        observations,
        *,
        search_config,
    ) -> "CodeDetectionResult":
        ...
```

ITS and EXP may require a richer interface because their detector compares observed tokens with a keyed random sequence rather than only binary observations. Do not distort those algorithms to fit a binary API; instead define a common WatermarkScheme abstraction above the sampler-specific detectors.

11.5 What the code layer receives

The relevant empirical channel should be summarized as something like:

```
intrinsic group mismatch
+ token substitutions caused by nucleotide substitutions
+ token insertions/deletions caused by retokenization
+ missing observations from cropping
+ local state mismatches from resynchronization mechanism
```

Only after measuring this effective channel should finite-length code parameters be selected.

12. Implementation specification

This section defines the first codebase at function level. Exact class names can change, but the invariants should not.

12.1 Baseline distribution adapter

```
def build_baseline_distribution(
    logits,
    *,
    baseline: Literal["B_tok", "B_bp"],
```

```

    temperature: float,
    top_k: int | None,
    top_p: float | None,
    valid_6mer_ids,
    special_token_ids,
) -> "BaselineDistribution":
    """Return the exact distribution that the experiment calls baseline.

    B_tok: processed categorical distribution over valid 6-mer tokens.
    B_bp: explicit representation of the released six-base-marginal mechanism.
    """

```

Required invariants for B_tok:

1. probabilities are non-negative;
2. sum over allowed tokens is 1 within numerical tolerance;
3. filtering/temperature is applied exactly once;
4. no special token is silently included unless the experiment defines it as part of the baseline;
5. the function records the pre/post-filter support size.

Tests:

- uniform synthetic logits;
- one dominant token;
- top-k=1;
- top-k larger than valid support;
- top-p boundary values;
- temperature 1;
- very small temperature;
- probability underflow;
- equivalence with frozen reference implementation on fixed logits.

12.2 Tokenizer and alignment utilities

```

def tokenize_aligned_dna(
    sequence: str,
    *,
    phase: int = 0,
    ambiguous: Literal["reject", "oov"] = "reject",
) -> list[int]:
    ...

```

Invariants:

- phase is in {0, 1, 2, 3, 4, 5};
- only complete 6-mers are returned;
- the mapping to official token IDs is frozen and tested;
- no hidden padding changes phase.

```

def tokenize_all_phases(sequence: str) -> dict[int, list[int]]:
    return {phase: tokenize_aligned_dna(sequence, phase=phase)
            for phase in range(6)}

```

12.3 Key derivation

Use a master key only as input to a standard KDF. Derive domain-separated subkeys:

```
K_master
|
+-- K_partition
+-- K_code
+-- K_alignment
+-- K_calibration (only if a keyed null procedure genuinely needs it)
```

Include a generation nonce or unique generation identifier where required to prevent reuse of an identical keyed random stream across many outputs.

Do not invent a bespoke cryptographic hash/KDF.

12.4 Construction-P partition function

```
def build_keyed_partition(
    valid_token_ids: Sequence[int],
    *,
    partition_key: bytes,
    state: bytes,
) -> "Partition":
    """Return a deterministic keyed 0/1 assignment reconstructable by detector."""
```

For the simplest stateless experiment, state can encode a public domain separator and generation position/code offset. For standalone detection after cropping, absolute generation position is problematic unless the detector searches offsets. This must be explicit in the scheme configuration.

A partition object should expose:

```
partition.group_of(token_id) -> 0 | 1
partition.group0_ids
partition.group1_ids
partition.partition_id
```

12.5 Exact binary coupling

```
def couple_bit_to_group(
    latent_bit: int,
    q: float,
    *,
    rng,
) -> int:
    """Maximally couple  $C \sim \text{Bern}(1/2)$  with  $Y \sim \text{Bern}(q)$ ."""
```

Reference behavior:

If $q \geq 1/2$:

$$P(Y = 1 \mid C = 1) = 1,$$

$$P(Y = 1 \mid C = 0) = 2q - 1.$$

If $q < 1/2$:

$$P(Y = 1 \mid C = 0) = 0,$$

$$P(Y = 1 \mid C = 1) = 2q.$$

Edge cases $q = 0$, $q = 1/2$, and $q = 1$ need explicit tests.

12.6 Token selection inside a partition

```
def sample_token_within_group(
    probs,
    *,
    partition: "Partition",
    group: int,
    rng,
) -> int:
    ...
```

If the selected group has baseline mass m_g , sample token z with

$$P(z \mid g) = \frac{P_t(z)}{m_g}$$

for tokens in that group.

If finite precision makes $m_g = 0$, do not silently fall back to the other group. The event and handling rule must be specified; with a true zero-mass group the coupling should never request that group under the mathematically correct rule.

12.7 Complete Construction-P sampler

```
def sample_watermarked_token_partition(
    probs,
    *,
    latent_bit: int,
    partition: "Partition",
    rng,
) -> tuple[int, dict]:
    q = probs[partition.group1_ids].sum()
    y = couple_bit_to_group(latent_bit, float(q), rng=rng)
    token_id = sample_token_within_group(
        probs, partition=partition, group=y, rng=rng
    )
    return token_id, {
        "q": float(q),
        "latent_bit": latent_bit,
        "observed_group": y,
        "intrinsic_error": int(y != latent_bit),
        "partition_id": partition.partition_id,
    }
```

Unit tests must compare empirical token frequencies against the input categorical distribution over millions of cheap synthetic draws.

12.8 ITS and EXP adapters

Implement ITS and EXP directly from [R6] and the authors' released code if available. The research code should expose a common entry point:

```
class WatermarkSampler(Protocol):
    def sample(self, probs, *, key_state, rng) -> tuple[int, dict]: ...
    def detector(self) -> "WatermarkDetector": ...
```

The experiment should never compare a carefully implemented Construction P against an approximate or incorrectly ported ITS/EXP implementation.

12.9 Observation function

For Construction P:

```
def observe_partition_symbol(
    token_id: int,
    *,
    partition: "Partition",
) -> int:
    return partition.group_of(token_id)
```

For ITS/EXP, the observation/statistic should be the one defined by their detector.

12.10 Standalone sequence detector

```
def detect_sequence(
    dna: str,
    *,
    verification_key: bytes,
    scheme: "WatermarkScheme",
    config: "DetectorConfig",
) -> DetectionResult:
    ...
```

The implementation should expose the complete hypothesis search so calibration can reproduce it exactly.

12.11 Baseline-vs-watermark integration seam

The experimental generation loop should look conceptually like:

```
logits, cache = model_step(tokens, cache)

baseline = build_baseline_distribution(
    logits[:, -1],
    baseline="B_tok",
    temperature=config.temperature,
    top_k=config.top_k,
    top_p=config.top_p,
    valid_6mer_ids=tokenizer.valid_6mer_ids,
    special_token_ids=tokenizer.special_token_ids,
)

if mode == "baseline":
    next_id = categorical_sample(baseline.probs, rng)
else:
```

```

next_id, wm_debug = wm_sampler.sample(
    baseline.probs,
    key_state=key_state,
    rng=rng,
)

```

For B_bp, keep a separately frozen wrapper around the released mechanism. Never label a direct-token experiment as behavior of the released `model.generate()` path unless the code actually uses that path.

13. Edge cases and protocol decisions

Every edge case below must have a deterministic behavior before large experiments start.

Case	Recommended first behavior	Reason
input length not divisible by 6	reject for generation E0-E4; detector tests phases	avoids silent <oo> generation behavior
lowercase DNA N or ambiguous IUPAC base	normalize to uppercase reject in clean generation; detector may mark affected token as erasure in later tests	representation-only difference no valid canonical 6-mer token
invalid character special token assigned nonzero mass	reject with location log explicitly; exclude/renormalize only if B_tok protocol defines DNA-only baseline	avoid hidden phase changes baseline must be unambiguous
top-k=1	treat as deterministic boundary	no exact-distortion sampler capacity
$q = 0$ or $q = 1$ in Construction P	valid but zero useful signal; coupling handles deterministically	avoid divide-by-zero
$q = 0.5$ fragment < 6 bp fragment shorter than detector minimum	no intrinsic group mismatch insufficient token evidence return insufficient_evidence, not not_watermarked	ideal partition mass no complete 6-mer avoid conflating no evidence with negative result
unknown phase unknown orientation circular sequence homopolymers / low complexity repeated 6-mer context	enumerate 0..5 forward + reverse complement local scan with wraparound include as explicit null category test for repeated keyed state correlations	bounded clean-crop ambiguity DNA representation coordinate zero arbitrary potential detector correlations may violate simple independent-null intuition
empty candidate group after truncation	should be impossible if groups defined on support and coupling is correct; raise/log if encountered	sampler correctness
extremely small probabilities	compute in stable floating point; record dtype	prevent underflow artifacts
tied logits	baseline processing defines tie behavior	reproducibility

A separate manifest should record these protocol decisions so that later code changes do not silently change the experiment.

14. Repository layout and reproducibility contract

The repository should separate the model adapter, watermark schemes, detector, edit simulator, statistics, and biological proxy analysis. This prevents the GENERator integration from becoming inseparable from one watermark construction.

```
generator-watermark/
├── README.md
├── pyproject.toml
├── environment.lock
├── configs/
│   ├── generator_v2_1p2b_euk.yaml
│   ├── generator_v2_1p2b_prok.yaml
│   ├── feasibility.yaml
│   ├── detector.yaml
│   └── full_eval.yaml
├── manifests/
│   ├── model_manifest.json
│   ├── tokenizer_manifest.json
│   └── experiment_protocol.json
├── src/generator_watermark/
│   ├── generator_adapter/
│   │   ├── model.py
│   │   ├── tokenizer.py
│   │   ├── distribution.py
│   │   ├── baseline_tok.py
│   │   ├── baseline_bp.py
│   │   ├── generation.py
│   │   └── trace.py
│   ├── watermark/
│   │   ├── interface.py
│   │   ├── key_schedule.py
│   │   ├── partition.py
│   │   ├── coupling.py
│   │   ├── its.py
│   │   ├── exp.py
│   │   ├── dipmark_adapter.py
│   │   └── protein_baseline_adapter.py
│   ├── coding/
│   │   ├── interface.py
│   │   ├── random_bits.py
│   │   ├── practical_ecc.py
│   │   └── prc_adapter.py
│   └── detector/
│       └── normalize.py
```

- orientation.py
- phase.py
- observe.py
- blocks.py
- alignment.py
- score.py
- calibration.py
- edits/
 - substitution.py
 - insertion.py
 - deletion.py
 - mixed.py
 - crop.py
 - reverse_complement.py
- stats/
 - entropy.py
 - power.py
 - nulls.py
 - multiple_testing.py
 - distinguishability.py
- biology/
 - composition.py
 - kmers.py
 - complexity.py
 - orfs.py
 - likelihood.py
- experiments/
 - E0_synthetic_sampler/
 - E1_generator_integration/
 - E2_channel_measurement/
 - E3_distribution_preservation/
 - E4_clean_detection/
 - E5_substitutions/
 - E6_indels/
 - E7_crop_phase_orientation/
 - E8_resynchronization/
 - E9_null_wrong_key/
 - E10_unkeyed_distinguishability/
 - E11_detector_query_removal/
 - E12_baselines/
 - E13_ecc_prc/
- tests/
 - test_tokenizer_roundtrip.py
 - test_valid_6mer_vocab.py
 - test_baseline_tok.py
 - test_baseline_bp.py
 - test_partition_balance.py
 - test_exact_marginals.py

```

├── test_phase_enumeration.py
├── test_reverse_complement.py
├── test_edit_accounting.py
├── test_calibration_reproducibility.py
├── test_no_key_stream_reuse.py
└── analysis/
    ├── make_channel_figures.py
    ├── make_detection_figures.py
    ├── make_edit_figures.py
    ├── make_null_figures.py
    ├── make_baseline_table.py
    └── build_report.py

```

14.1 Reproducibility manifest

Every generated cohort should be traceable to a manifest containing at least:

```

{
  "model_repo": "GenerTeam/...",
  "model_revision": "<exact revision>",
  "tokenizer_revision": "<exact revision>",
  "transformers_version": "<exact>",
  "torch_version": "<exact>",
  "cuda_version": "<exact>",
  "baseline": "B_tok",
  "temperature": 1.0,
  "top_k": null,
  "top_p": 1.0,
  "watermark_scheme": "partition_v1",
  "detector_version": "v1",
  "key_id": "non-secret fingerprint only",
  "generation_nonce_policy": "...",
  "dtype": "bfloat16",
  "seed_manifest": "..."
}

```

Never write the raw secret key into experiment metadata intended for publication.

14.2 Freeze external code paths

The source audit found model-specific generation behavior. Therefore the experiment should vendor or pin the minimal relevant files/revisions rather than allowing a later `pip install -U transformers` to change processor order or generation semantics.

For every release, keep a small reference test whose expected output is a cryptographic digest of:

- valid 6-mer token ID list;
- special-token list;
- selected fixed logits -> processed B_tok probabilities;
- selected fixed logits -> B_bp marginal distributions.

If that digest changes, the baseline has changed and old/new results must not be pooled silently.

15. Stable experiment IDs and project dependency graph

Use the same IDs in code, filenames, plots, lab notes, and manuscript drafts.

ID	Experiment	Main question	Main output
E0	Synthetic keyed-sampler validation	Is the sampler mathematically/implementationally correct?	exact-marginal tests + channel diagnostics
E1	GENERator integration and baseline audit	Are B_{tok} and B_{bp} implemented exactly?	frozen integration trace
E2	Native 6-mer channel measurement	How much stochastic/keyed signal exists under real generation?	generator_channel.parquet
E3	Distribution-preservation test	Does watermarked generation preserve the declared baseline?	marginal/fidelity tests + unkeyed preliminary classifier
E4	Clean standalone detection	How much DNA is needed for DNA+key detection?	TPR-vs-length at fixed FPR
E5	Nucleotide substitutions	How do substitutions map into token/detector errors?	substitution curves
E6	Insertions/deletions	How severely does phase loss affect detection?	indel curves + token-channel traces
E7	Cropping/orientation/phase	Can clean fragments be detected without original coordinate/strand?	phase/orientation curves
E8	Resynchronization	Which S0-S3 method confines indel damage?	local recovery plots
E9	Null and wrong-key calibration	Are system-level false positives controlled?	null distributions + thresholds
E10	Unkeyed distinguishability	Can an observer identify watermarked sequences without key?	AUC / classifier diagnostics
E11	Detector-query removal	How many generic edits are needed to remove detection?	score-vs-edit-budget curves
E12	Baseline comparison	Do generic keyed watermark methods already solve the use case?	comparison table
E13	ECC/PRC integration	What coding/security layer fits the measured channel?	code parameters + conditional/formal claims

15.1 Dependency graph

```

E0  synthetic sampler correctness
 |
 v

```



The project should not spend weeks on E13 before E2-E4 succeed.

16. Data schemas

A reproducible project should define its datasets before generating large cohorts.

16.1 generator_channel.parquet

One row per generated/model-evaluated token position.

```

run_id
sequence_id
prompt_id
checkpoint
domain

```

```

model_revision
tokenizer_revision
baseline
position_token
position_bp
context_tokens
context_bp
temperature
top_k
top_p
do_sample
entropy_bits
max_probability
support_size
effective_vocab_inverse_simpson
effective_vocab_entropy
valid_6mer_mass
special_token_mass
partition_id
q
abs_q_minus_half
predicted_intrinsic_mismatch
information_token_bits
information_bp_bits
latent_bit
observed_group
selected_token_id
selected_6mer
selected_token_probability
selected_rank
rng_stream_id

```

Definitions:

$$N_{\text{eff},2} = \frac{1}{\sum_z P_t(z)^2}$$

and optionally entropy effective vocabulary

$$N_{\text{eff},H} = 2^{H(P_t)}$$

when entropy is measured in bits.

Store both because they answer different questions and the random-partition variance result naturally involves $N_{\text{eff},2}$.

16.2 generation_results.parquet

One row per complete generated sequence.

```

run_id
sequence_id
prompt_id
checkpoint
baseline
watermark_scheme

```

```
key_id
length_tokens
length_bp
temperature
top_k
top_p
mean_logprob
median_logprob
gc_fraction
invalid_token_count
special_token_count
runtime_seconds
peak_gpu_memory_mb
output_fasta_path
trace_path
```

16.3 detection_results.parquet

```
run_id
sequence_id
is_watermarked
source_class
watermark_scheme
key_id
is_correct_key
length_bp
orientation_tested
phase_tested
window_start_bp
window_end_bp
code_offset
score
nominal_p_value
global_p_value
threshold
decision
best_orientation
best_phase
best_window_start_bp
best_code_offset
n_hypotheses
n_tokens_used
attack_type
attack_rate_nucleotide
edit_count
induced_token_substitutions
induced_token_insertions
induced_token_deletions
recovered_alignment_fraction
```

16.4 edit_results.parquet

```
sequence_id
edit_seed
```

```
edit_type
requested_rate
original_length_bp
edited_length_bp
substitution_count
insertion_count
deletion_count
crop_fraction
orientation
original_token_count
best_phase
aligned_token_matches
induced_token_error_rate
wm_observation_substitution_rate
wm_observation_indel_rate
detection_score
global_p_value
decision
```

16.5 null_results.parquet

```
sequence_id
null_family
source_dataset
taxon_or_context
length_bp
key_id
orientation_phase_window_searches
max_score
global_p_value
decision
```

Null families should include natural DNA, ordinary GENERator output, wrong keys on watermarked output, shuffled composition-preserving DNA, low-complexity sequences, and repetitive sequences.

17. E0 - synthetic sampler validation

E0 is CPU-only and should be completed before loading GENERator weights.

17.1 Question

Does each keyed sampler reproduce the probability distribution it claims to reproduce, and does its observed detector channel match its mathematical prediction?

17.2 Synthetic distributions

At minimum test:

```
D1 uniform over 4096 tokens
D2 one dominant token: p1=.90, remaining .10 spread
D3 effective support ~4
D4 effective support ~16
```

```
D5 effective support ~64
D6 effective support ~256
D7 flat head + long tail
D8 top-k truncated support = 2
D9 top-k truncated support = 4
D10 nearly deterministic p1=.999
```

For Construction P, generate multiple independent keyed partitions per distribution.

17.3 Monte Carlo scale

For inexpensive distributions, use at least 10^6 and preferably 10^7 draws per key/configuration when testing marginal correctness. This is cheap enough to expose subtle implementation errors.

17.4 Measurements

For every token z with non-negligible target probability, estimate

$$\hat{P}_{\text{WM}}(z)$$

and compare with

$$P(z).$$

Record:

- maximum absolute probability error;
- total variation distance;
- chi-square or other diagnostic where expected counts permit;
- q distribution;
- empirical intrinsic mismatch;
- predicted $|q - 1/2|$;
- empirical $I(C; Y)$;
- runtime per million samples.

Do not use a statistical hypothesis-test p-value as the only correctness criterion. With millions of samples, tiny irrelevant implementation differences can become statistically significant; the essential question is whether errors scale like Monte Carlo noise and vanish with sample size.

17.5 Exact-marginal regression test

For fixed distribution P and truly random latent bits, test at sample sizes

$$N \in \{10^4, 10^5, 10^6, 10^7\}.$$

Plot total variation versus N . A correct Monte Carlo implementation should show the expected sampling-error decay rather than a fixed offset.

17.6 Edge-case tests for Construction P

$$q = 0.5$$

Expected intrinsic mismatch:

0.

$q = 0.9$

Expected mismatch:

0.4.

$q = 1$

The observed group is always 1, and the latent bit is unrecoverable half the time. No divide-by-zero is allowed.

$q = 0$

Symmetric.

17.7 Continue condition

Continue only when:

1. marginal error is consistent with Monte Carlo error across all synthetic distributions;
2. empirical mismatch matches the analytic coupling prediction;
3. ITS/EXP reference tests match published/reference implementation behavior on fixed toy distributions;
4. deterministic/top-k=1 cases correctly show no useful exact-distortion watermark signal.

Any persistent marginal shift under true-random latent bits is an implementation bug, not a tolerable research result.

18. E1 - GENERator integration and exact baseline audit

18.1 Question

Can we obtain and freeze the exact token probability distribution used by the intended experiment without accidentally invoking GENERator-v2's base-marginal logits processor?

18.2 Tasks

1. Load the development checkpoint.
2. Load its remote/custom tokenizer and model code under a pinned revision.
3. Enumerate all tokenizer entries.
4. Verify exactly 4096 canonical A/T/C/G 6-mers in the expected set.
5. Identify and record all special tokens.
6. Verify prompt round-trip at lengths divisible by six.
7. Verify expected <oov> behavior for deliberately misaligned prompts.
8. Run the official/released generation path and trace `_BPLogitsProcessor` inputs/outputs.
9. Implement `B_tok` by intercepting the processed token distribution before `_BPLogitsProcessor` performs base marginalization.
10. Keep a separate `B_bp` wrapper that reproduces released behavior.

18.3 Reference trace

For a fixed prompt and fixed RNG seed, save for 10-20 generation steps:

```
raw top logits
valid 6-mer IDs
probabilities before truncation
probabilities after temperature/top-k/top-p
B_tok selected token
B_bp six base marginals
B_bp selected bases
B_bp reconstructed token
```

This trace becomes a regression fixture.

18.4 Important processor-order test

Do not rely only on source comments stating that `_BPLogitsProcessor` is appended last. Under the pinned Transformers version, instrument the actual list/order of logits processors/warpers and verify where temperature, top-k, and top-p act relative to `_BPLogitsProcessor`.

The paper must be able to state exactly what distribution is being watermarked.

18.5 Baseline equivalence test

Implement an ordinary categorical sampler on `B_tok` and verify that disabling the watermark but using the custom loop gives the same distribution as direct sampling from the captured `B_tok` probabilities.

Separately verify `B_bp` against the released `model.generate()` path.

Do not expect individual generated sequences to match when stochastic sampling is used; compare deterministic traces where possible and distributions otherwise.

18.6 Deliverables

```
manifests/model_manifest.json
manifests/tokenizer_manifest.json
experiments/E1_generator_integration/reference_trace.jsonl
experiments/E1_generator_integration/vocab.csv
experiments/E1_generator_integration/processor_order.txt
experiments/E1_generator_integration/baseline_equivalence.csv
```

18.7 Continue condition

Proceed to E2 only when the team can answer, in one sentence each:

- What exactly is B_{tok} ?
 - What exactly is B_{bp} ?
 - Which one is the watermark target?
 - Where is the probability vector captured?
 - Which operations have already been applied to it?
-

19. E2 - measure the real 6-mer sampling channel

E2 is the highest-value feasibility experiment.

19.1 Question

Does real GENERator-v2 generation expose enough uncertainty over 6-mer tokens for a keyed sampler to create detectable evidence at useful DNA lengths?

The nominal vocabulary size 4096 is not the answer. The actual distribution may be highly concentrated or aggressively truncated.

19.2 Model selection

Start with a current 1.2B GENERator-v2 checkpoint for development. Use one domain first, then confirm on the second domain if the project aims to generalize across them. Use a 3B checkpoint only after the pipeline is stable.

Record the exact model revision.

19.3 Prompt/context selection

Use benign public genomic contexts appropriate to the checkpoint. Include multiple context categories rather than one convenient sequence. The objective is to characterize the model, not optimize any biological function.

At minimum vary:

- taxonomic/context source;
- prompt length;
- generated position within continuation;
- decoding configuration.

Keep a prompt manifest so later experiments can use held-out prompt splits.

19.4 Decoding configurations

First reproduce the model's documented/released generation settings. Then run a controlled sweep.

Candidate sweep to adapt after E1:

```
temperature: 0.5, 0.7, 1.0
```

```
top_k:  
  none  
  8  
  16  
  32  
  64
```

```
top_p:  
  1.0  
  0.95  
  0.90
```

Do not blindly run the full Cartesian product. Pick a small set that spans deterministic/concentrated to stochastic while remaining plausible generation policies.

Include top-k=1/greedy as a zero-capacity control.

19.5 Target scale

Initial debugging:

$$\geq 100,000$$

token positions.

Preferred feasibility dataset:

$$500,000 \text{ to } 1,000,000$$

token positions across contexts/configurations, if compute allows.

Because each token corresponds to six bases, one million token positions represents six million generated/evaluated base positions.

19.6 Per-position metrics

For every position compute:

Shannon entropy

$$H_t = - \sum_z P_t(z) \log_2 P_t(z).$$

Top-1 probability

$$\max_z P_t(z).$$

Effective vocabulary

$$N_{\text{eff},2} = \frac{1}{\sum_z P_t(z)^2}.$$

Keyed partition balance

$$q_t = \sum_{z: g_K(z)=1} P_t(z),$$

$$|q_t - 1/2|.$$

Construction-P intrinsic mismatch

$$e_t = |q_t - 1/2|.$$

Information per token

$$I_t = h_2(q_t) - \frac{1}{2} h_2(|2q_t - 1|).$$

Information per base

$$I_{t,\text{bp}} = I_t/6.$$

Also evaluate ITS/EXP-specific expected/detector statistics if they can be estimated from stored distributions/key draws.

19.7 Random-key versus oracle upper bound

For Construction P compare:

Detector-compatible random keyed partition

Partition is independent of P_t and reconstructable from token/key.

Oracle probability-balanced partition

Use model probabilities to find a partition with mass close to $1/2$.

The oracle is not necessarily a deployable standalone detector. It answers a diagnostic question:

Is the limiting factor the random keyed partition, or the underlying token distribution itself?

If oracle performance is dramatically better, a decoder-compatible adaptive partition becomes a separate design problem.

19.8 Main plots

1. entropy histogram;
2. top-1 probability histogram;
3. effective-vocabulary histogram;
4. $|q - 1/2|$ histogram for random keyed partitions;
5. random vs oracle partition balance;
6. information/token;
7. information/base;
8. cumulative expected information versus DNA length;
9. information versus effective vocabulary;
10. information versus top-1 probability;
11. breakdown by prompt/context;
12. breakdown by decoding configuration;
13. B_{tok} versus diagnostics induced by B_{bp} .

19.9 Interpretation examples

These are illustrative decision interpretations, not predicted results.

Example A

If mean information is

0.8 bits/token

then that corresponds to

0.133 bits/bp.

The clean channel likely has ample statistical signal; attention should move quickly to edit synchronization and baseline comparison.

Example B

If mean information is

0.12 bits/token

then

0.02 bits/bp.

The signal may still be usable over kilobase windows, but finite-length simulation is necessary.

Example C

If the distribution is effectively top-1 at most positions and clean detection simulations require tens or hundreds of kilobases, the 4096-token vocabulary is not providing the hoped-for practical advantage.

19.10 Feasibility gate

Do not use one arbitrary information threshold as a theorem. Continue if either:

1. measured information per base is clearly non-negligible **and** finite-length clean simulations predict useful detection within the intended fragment size; or
2. ITS/EXP directly demonstrate acceptable clean detection even if the binary partition diagnostic looks weaker.

A practical project-management warning is:

- if mean $I_{bp} < 0.01$ **and** no tested scheme predicts/demonstrates useful detection by 5 kb, redesign decoding/sampler choice before proceeding.

A positive early signal is:

- mean $I_{bp} \geq 0.05$, or clean model-free detection simulations indicating the target operating point by ≤ 5 kb.

These numbers are internal gates, not literature guarantees.

20. E3 - distribution-preserving generation on GENERator

20.1 Question

Does the implementation preserve the exact declared baseline distribution when the sampler construction says it should?

This experiment has two levels:

1. **conditional correctness** on repeated fixed probability vectors;
2. **sequence-level behavior** under autoregressive feedback.

20.2 Fixed-state test using real GENERator distributions

Sample thousands of real P_t vectors collected in E2. For each vector, hold it fixed and draw many tokens with:

- baseline categorical sampler;
- Construction P with independent fair latent bits;
- ITS/EXP according to their exact construction.

For methods with exact distribution-preservation claims, empirical token frequencies should converge to the same P_t .

This isolates sampler correctness from autoregressive changes in future contexts.

20.3 Autoregressive matched-cohort test

Generate matched cohorts from the same prompt distribution using:

- B_{tok} ordinary sampling;
- watermarked B_{tok} for each sampler;
- B_{bp} control.

Do not compare B_{tok} watermarked output only against B_{bp} . They are different generators by construction.

20.4 Distribution metrics

At token level:

- token unigram frequencies;
- top-token/rank distributions;
- token entropy along generation;
- 6-mer frequency;
- sequence log-likelihood under an independent/frozen scorer where appropriate.

At nucleotide level:

- A/C/G/T composition;
- GC;
- dinucleotide frequencies;
- k -mers for reasonable k ;
- run-length distribution;
- sequence complexity;
- repeat statistics.

Do not claim exact sequence-distribution equality solely because these finite-dimensional summaries match. They are empirical diagnostics. Exact equality, where claimed, comes from the sampler proof relative to the declared baseline.

20.5 Unkeyed preliminary distinguisher

Train a simple classifier:

```
baseline B_tok  
vs  
watermarked B_tok
```

Features:

- nucleotide mono/di/ k -mers;
- 6-mer counts;
- GC;
- run lengths;
- simple complexity statistics;
- optional independent model score.

Split by prompt, generation seed/nonce, and watermark key. Do not allow the train/test split to contain the same prompts or key-specific artifacts.

Internal gate:

$$\text{AUC} \leq 0.55$$

is a reasonable “no obvious empirical distinguisher” target for the first simple classifiers. It is **not** a cryptographic undetectability proof.

20.6 Runtime overhead

Measure:

- tokens/s;
- bases/s;
- sampler CPU/GPU time;
- key/partition generation time;
- extra memory;
- detector-independent generation overhead.

A 4096-way partition sum can be vectorized. If watermark overhead is a meaningful fraction of Transformer inference time, profile before optimizing theory.

20.7 Continue condition

Continue if:

1. fixed-state exact-marginal tests pass;
2. autoregressive cohorts show no unexplained systematic shift beyond what follows from comparing distinct baselines;
3. simple unkeyed classifier remains near chance;
4. runtime overhead is manageable relative to generation.

If Construction P with true random latent bits creates a persistent marginal shift relative to B_{tok} , stop and fix the sampler.

21. E4 - clean standalone detection

21.1 Question

Given only clean generated DNA and the verification key, how much sequence is needed to distinguish watermarked from null output at a fixed false-positive rate?

This is the first end-to-end test of the central use case.

21.2 Detector variants

Implement separately:

- Construction-P binary detector;
- ITS detector from [R6];
- EXP detector from [R6];
- any direct protein-watermark adaptation used as a baseline;
- DiPmark-style detector if implemented.

Do not force every method to use one generic score if its original detector is different.

21.3 Sequence lengths

Use multiples of six:

bases	6-mer tokens
120	20
240	40
480	80
960	160
1,920	320
3,840	640
7,680	1,280
15,360	2,560

Add shorter or longer values after the first curve reveals the transition region.

21.4 Nulls

At minimum:

1. ordinary B_{tok} generations;
2. watermarked generations tested under wrong keys;
3. natural DNA fragments of matched length/context;
4. B_{bp} generations as a model-specific control.

The final detector may need separate calibration by null family or a conservative pooled null. Do not choose after looking only at the most favorable one.

21.5 Operating point

The initial feasibility operating point is

$$\text{FPR} \leq 10^{-3}.$$

This is intentionally modest enough to estimate directly. Later forensic/deployment claims need much lower system-level FPR.

21.6 Report

For each method and length:

- TPR at fixed FPR;
- ROC and precision-recall where meaningful;

- score distribution;
- wrong-key distribution;
- phase/orientation behavior;
- runtime;
- confidence intervals.

21.7 Internal continue threshold

At system FPR $\leq 10^{-3}$:

- TPR ≥ 0.95 by 1-2 kb: excellent clean channel;
- TPR ≥ 0.95 by 5 kb: sufficient to continue;
- 10-15 kb required: investigate whether the use case still makes sense before adding complex synchronization.

These are project-management thresholds, not published guarantees.

21.8 Important outcome: generic sampler already wins

If ITS or EXP directly gives better clean detection and exact distortion-free behavior than Construction P with no genome-specific disadvantage, use it. The objective here is not to preserve a home-grown construction.

22. E5 - nucleotide substitution experiment

22.1 Question

How do nucleotide substitutions translate into 6-mer token changes and detector degradation when phase is preserved?

22.2 Rates

Test independently:

$$p \in \{0.001, 0.005, 0.01, 0.02, 0.05\}.$$

Use a defined substitution model. Start with uniform replacement among the other three bases. Later add a transition-biased benign mutation model if scientifically appropriate.

22.3 Measurements

For every edited sequence record:

- number/fraction of nucleotide substitutions;
- fraction of original aligned 6-mers changed;
- fraction of watermark observations changed;
- detection score;
- TPR at fixed system FPR;
- token-level error burst distribution.

Compare empirical changed-token fraction with

$$1 - (1 - p)^6.$$

Differences can arise because multiple substitutions land in the same token.

22.4 Project gate

At 5 kb and $\text{FPR} \leq 10^{-3}$, a useful first target is:

$$\text{TPR} \geq 0.90$$

after 1% independent nucleotide substitutions.

If this fails even though clean detection is excellent, investigate the detector/code layer before synchronization: substitutions do not destroy global phase.

23. E6 - nucleotide insertions and deletions

23.1 Question

How quickly does a global 6-mer token detector fail after nucleotide indels, and what effective token edit channel do those nucleotide events create?

23.2 Separate insertion and deletion tests

For each:

$$r \in \{0.001, 0.005, 0.01, 0.02, 0.05\}.$$

Define the simulation process precisely. For example, “1% insertion rate” must specify whether one insertion event is attempted independently per original base or exactly 0.01 L insertion events are placed uniformly.

Use exact realized edit counts in analysis.

23.3 Mixed edits

After separate curves, test mixed conditions such as:

0.5% substitutions
0.25% insertions
0.25% deletions

then scale upward.

23.4 Metrics beyond TPR

For every sequence compute:

- correct original phase before first edit;
- phase modulo six as edits accumulate;
- length of correctly aligned token runs;

- induced token insertion/deletion/substitution counts under an alignment;
- detector score locally around each edit;
- fraction of sequence recoverable by S0/S1/S2/S3.

This experiment should explain failure, not merely state that TPR fell.

23.5 S0 baseline expectation

The one-global-phase detector is expected to degrade sharply as soon as a non-multiple-of-six net indel occurs. Treat that as a baseline characterization, not a surprise.

23.6 Internal target after synchronization work

At 5 kb and $\text{FPR} \leq 10^{-3}$, an exploratory target of

$$\text{TPR} \geq 0.85$$

under 1% total nucleotide indel load is enough to justify continuing the current synchronization design.

If all S1-S3 methods remain near chance at much smaller indel loads, move to “CONTINUE BUT REDESIGN SYNCHRONIZATION” or stop the edit-survival claim.

24. E7 - cropping, phase, orientation, and local fragments

24.1 Question

Can a clean partial DNA fragment be verified without knowing its original coordinate or strand?

24.2 Cropping fractions

From full generated sequences retain:

$$\{75\%, 50\%, 25\%, 10\%\}$$

using multiple start positions.

Also test fixed absolute fragment lengths so results are not confounded by different original sequence lengths.

24.3 Phase tests

For every crop, deliberately hide the original offset and enumerate all six phases. Record:

- score of correct phase;
- best incorrect phase;
- margin;
- rank of correct phase;
- global calibrated detector score.

24.4 Orientation

Test both original and reverse-complement forms. The detector should return the best orientation/phase but the p-value must reflect the full search.

24.5 Success condition

Clean cropping should be substantially easier than indel synchronization. If a several-kb clean crop cannot be detected despite good full-sequence clean detection, debug phase/key-offset search before proceeding.

25. E8 - resynchronization comparison

25.1 Question

Can the detector confine the effect of nucleotide indels to local regions without access to GENERator logits?

25.2 Compare

- S0 global phase;
- S1 independent windows/blocks;
- S2 local sequence-derived state;
- S3 phase-aware edit alignment.

Only add S4 after observing the failure pattern.

25.3 Matched experiment

Use the same edited sequences for every synchronization method. This avoids comparing different random edit realizations.

25.4 Primary plots

1. TPR vs nucleotide indel rate;
2. detector score vs indel rate;
3. token error rate vs indel rate;
4. local observation error vs distance from nearest edit;
5. recovered alignment fraction;
6. runtime vs fragment length;
7. number of tested hypotheses;
8. block size vs detection/correction tradeoff.

25.5 Block-size sweep

Candidate token-aligned block sizes:

{120, 240, 480, 960, 1920, 3840} bp.

Do not select a block smaller than clean detection can support at the intended FPR.

25.6 Decision

If S1 or S3 recovers useful performance at moderate indel rates, avoid adding theoretical synchronization machinery prematurely.

If no standalone method can recover phase without the original generation trace, the central “DNA + key only” requirement may need to be weakened or the project should be reframed.

26. E9 - null distributions, wrong keys, and system-level false positives

False positives are a system property, not just a per-window property.

26.1 Basic definitions

True-positive rate:

Among genuinely watermarked samples under the evaluated condition, what fraction are called watermarked?

Example:

```
1,000 watermarked fragments
950 detected
TPR = 95%
```

False-positive rate:

Among samples that do not carry the tested watermark/key, what fraction are incorrectly called watermarked?

Example:

```
1,000,000 null fragments
10 detections
FPR = 10 / 1,000,000 = 10^-5
```

26.2 Multiple search hypotheses

A forensic scan may test:

```
100,000 sequences
× 2 orientations
× 6 phases
× 20 windows
```

for

$$M = 100000 \times 2 \times 6 \times 20 = 24,000,000$$

nominal hypotheses before code offsets or multiple keys are considered.

A simple conservative union bound is

$$P(\text{at least one FP}) \leq M\alpha,$$

where α is a per-hypothesis false-positive probability.

At this scale, $\alpha = 10^{-3}$ is obviously unsuitable for deployment-wide scanning even though it is useful for early experiments.

26.3 Preferred calibration strategy

Define one **global test statistic** such as the maximum score over all allowed orientations/phases/windows/offsets. Then generate the null distribution of that maximum using the exact same search procedure.

This automatically incorporates dependencies among overlapping windows better than treating all hypotheses as independent in a Bonferroni calculation.

26.4 Null families

Use at least:

1. ordinary B_{tok} output;
2. ordinary B_{bp} output;
3. natural genomic DNA;
4. watermarked DNA tested under wrong keys;
5. composition-preserving shuffled DNA;
6. repetitive DNA;
7. low-complexity DNA;
8. multiple genomic contexts/taxa appropriate to the project.

The detector should not be calibrated only on the easiest null.

26.5 Wrong-key sweeps

For each watermarked sequence, test many unrelated keys. This evaluates:

- cross-key score distribution;
- accidental key collisions;
- whether repeated token patterns produce anomalous scores for some keys;
- multiple-key scanning cost.

26.6 Empirical limits

If zero false positives are seen in N independent null trials, a rough 95% upper confidence bound is

$$\frac{3}{N}.$$

Therefore:

- around 10^{-3} : thousands of nulls can characterize it;
- around 10^{-6} : millions are needed;
- around 10^{-9} : billions would be required for direct empirical validation.

Do not claim an empirically measured 10^{-9} FPR from a small corpus.

26.7 Later low-FPR path

For stricter claims combine:

- analytic null result where justified;
- permutation/key-randomization calibration;
- millions of cheap null windows;
- formal PRC soundness if E13 supplies it;
- explicit correction for the complete search.

27. E10 - unkeyed distinguishability

27.1 Question

Can an observer who lacks the key distinguish watermarked B_{tok} output from ordinary B_{tok} output?
This is an empirical attack, not a substitute for a cryptographic proof.

27.2 Data split

Hold out simultaneously:

- prompts/contexts;
- generation nonces/seeds;
- watermark keys;
- if possible, genomic source categories.

A classifier that memorizes a key or prompt is not measuring general watermark distinguishability.

27.3 Classifiers

Start simple:

- logistic regression on nucleotide k -mers;
- logistic regression on 6-mer counts;
- gradient-boosted trees on summary statistics;
- independent model likelihood/embedding classifier only if simple methods reveal nothing and compute permits.

27.4 Report

- AUC with confidence interval;
- accuracy at balanced classes;
- calibration;
- feature importance for interpretable models;
- cross-key generalization;
- cross-prompt generalization.

Internal target:

$$\text{AUC} \leq 0.55$$

for the simple unkeyed distinguishers.

If a simple 6-mer histogram classifier reaches high held-out AUC, the distribution-preservation or key-schedule implementation needs investigation.

28. E11 - detector-query removal

This is a defensive removal-resistance evaluation.

28.1 Attacker knowledge

Default A3 setting:

The attacker knows:

```
model family
watermark algorithm
6-mer tokenization
block/window policy
public detector behavior
```

but not:

```
secret verification/generation key
private keyed random stream
```

Two detector APIs:

1. binary yes/no;
2. numerical score.

The score API is more informative to an attacker and should be treated separately.

28.2 Safe objective

Use only generic DNA sequence edits. The objective is:

reduce detector score while keeping generic sequence statistics close to the original.

Do not optimize pathogenicity, biological fitness, functional enhancement, or any harmful biological property.

28.3 Measurements

For each attack algorithm/budget:

- fraction of nucleotide edits;
- substitutions/insertions/deletions separately;
- detector queries used;
- original and final score;
- success rate;
- GC/k-mer/independent-likelihood change;
- phase/synchronization effect.

28.4 Redesign signal

If a generic score-query attacker reliably removes detection with fewer than 1% nucleotide edits while generic sequence-quality proxies remain nearly unchanged, the practical removal-resistance claim is weak and should be redesigned or narrowed.

This is an internal decision threshold, not a universal watermark-security definition.

29. E12 - baseline comparison

Novelty is not the objective, but fair baselines are necessary to decide whether custom engineering is worthwhile.

29.1 Required methods where implementation is feasible

1. Construction P: keyed binary 6-mer partition + coupling.
2. Kuditipudi ITS [R6].
3. Kuditipudi EXP [R6].
4. DiPmark-style distribution-preserving method [R7].
5. Direct adaptation of the private-key protein watermark method [R8].
6. A simple green/red keyed token watermark as a distortion-allowing reference.
7. E13 ECC/PRC version once available.

DNAMark/CentralMark [R13] is relevant biological watermarking context, but it is not necessarily a direct token-level keyed-sampler baseline.

29.2 Comparison table

Use the same B_{tok} , prompts, lengths, and edit realizations wherever the method permits.

Method	Exact/expected distribution property	DNA+key standalone?	Clean TPR @ fixed FPR	1% substitutions	1% indels	50% crop	unkeyed AUC	runtime
Partition coupling	target exact	yes	TBD	TBD	TBD	TBD	TBD	TBD
ITS	marginal per [R6]	per [R6]	TBD	TBD	TBD	TBD	TBD	TBD
EXP	per [R6]	per [R6]	TBD	TBD	TBD	TBD	TBD	TBD
DiPmark adaptation	verify	verify	TBD	TBD	TBD	TBD	TBD	TBD
Protein-WM adaptation	verify	yes in original protein setting	TBD	TBD	TBD	TBD	TBD	TBD
simple green/red	generally distorts	yes	TBD	TBD	TBD	TBD	TBD	TBD

Do not fill “yes” for a property simply because a related paper has it. Verify the exact adaptation.

29.3 Acceptable outcome: use generic method directly

If EXP/ITS or another existing method transfers cleanly and outperforms the custom partition design, the correct technical decision is:

USE GENERIC WATERMARK DIRECTLY

and focus engineering effort on 6-mer phase/edit handling.

30. E13 - ECC and PRC integration

30.1 Question

After measuring the real token/edit channel, what coding layer provides the needed finite-length error tolerance and, if desired, cryptographic pseudorandomness?

30.2 Conventional ECC first

Use a practical code to establish whether the measured channel is correctable at relevant lengths. Choose based on:

- substitution rate;
- insertion/deletion structure;
- erasure opportunities;
- block size;
- decoding runtime;
- code rate.

Do not select code rate before E5-E8 quantify the effective observation channel.

30.3 PRC literature table

Maintain a living table:

Construction	Alphabet	Edits	Random/adversarial	Security assumption	Cropping	Public implementation	Practical finite-length status here
Christ & Gunn 2024 [R10]	see paper	substitutions/deletions in stated settings	stated channel	stated cryptographic assumptions	relevant LM construction	verify	TBD
Golowich & Moitra 2024 [R14]	construction-dependent	insertion/deletion	stated model	stated assumptions	verify	verify	TBD
Christ et al. 2026 [R11]	binary result available	worst-case edits in stated model	adaptive/worst-case properties	stated assumptions	verify	verify	TBD
Huang et al. 2026 [R12]	binary/public key results	edit errors	stated channel	see paper	verify	verify	TBD

The final document/manuscript must quote guarantees accurately from the papers rather than compressing all rows into “PRCs tolerate edits.”

30.4 Composition target

For a sampler with exact distribution preservation under truly uniform latent randomness, a possible proof target is:

Let G be the frozen efficient autoregressive B_{tok} generator. Let S be an efficient watermark sampler such that replacing the secret latent code by independent uniform randomness makes every conditional

emitted-token distribution exactly equal to G . Let C_K be computationally indistinguishable from the required uniform random source.

Then a distinguisher between final ordinary and watermarked DNA may be convertible into a distinguisher between C_K and uniform, with care for:

- adaptive autoregressive contexts;
- stateful key schedules;
- generation nonce reuse;
- block resets;
- detector-independent sampler state;
- finite precision/implementation deviations.

This is a proof obligation, not a result established by this plan.

31. One-week feasibility sprint

The objective is to decide whether the project deserves a longer implementation before investing in PRCs or complicated synchronization.

Day 1 - E1 integration and tokenizer audit

Question

Can we reproduce both B_{tok} and B_{bp} exactly enough for controlled experiments?

Code

- tokenizer manifest script;
- baseline distribution adapter;
- direct token generation loop;
- released B_{bp} wrapper;
- trace logger.

Work

1. install/pin GENERator-v2 development checkpoint;
2. enumerate 4096 valid 6-mers + special tokens;
3. test aligned and misaligned prompt behavior;
4. instrument `_BPLogitsProcessor`;
5. freeze processor order;
6. generate 20 short reference sequences with each baseline;
7. benchmark speed/memory.

Outputs

```
manifests/model_manifest.json
manifests/tokenizer_manifest.json
experiments/E1_generator_integration/reference_trace.jsonl
experiments/E1_generator_integration/runtime.csv
```

Stop/redesign

If direct B_{tok} cannot be defined reproducibly from the model outputs, the token-level watermark experiment is not yet well-defined.

Day 2 - E0 completion + E2 channel measurement

Question

Does the true 6-mer token distribution contain enough usable uncertainty?

Code

- Construction-P sampler;
- ITS/EXP reference adapters or at least key-statistic simulator;
- entropy/effective-vocabulary logger.

Work

1. finish 10^6 - 10^7 synthetic E0 tests;
2. collect $\geq 100,000$ real token distributions;
3. target 500k-1M if throughput permits;
4. compute entropy, effective support, q , intrinsic mismatch, information/token/base;
5. compare random keyed vs oracle-balanced partitions;
6. compare realistic decoding settings.

Outputs

```
experiments/E2_channel_measurement/generator_channel.parquet
analysis/q_distribution.pdf
analysis/effective_vocab.pdf
analysis/information_per_token.pdf
analysis/information_per_bp.pdf
analysis/cumulative_information.pdf
```

Decision

If B_{tok} is effectively deterministic under realistic decoding and clean finite-length simulation is poor, do not proceed merely because the nominal vocabulary is 4096.

Day 3 - E3 exact sampler on real generation

Question

Does the watermarked generation implementation preserve its declared baseline?

Work

- fixed-state real- P_t marginal test;
- matched prompt cohorts;
- preliminary unkeyed classifier;
- runtime benchmark.

Outputs

```
experiments/E3_distribution_preservation/fixed_state_marginals.parquet
experiments/E3_distribution_preservation/sequence_metrics.parquet
experiments/E3_distribution_preservation/unkeyed_auc.json
```

Continue

Only if exact-marginal tests pass and no obvious held-out unkeyed distinguisher appears.

Day 4 - E4 clean standalone detector

Question

Can DNA + key alone detect the watermark at useful length?

Code

- forward/reverse-complement enumerator;
- six-phase tokenizer;
- scheme-specific detector;
- window/offset search;
- empirical null calibration.

Work

Test 120 bp through 15.36 kb, with dense sampling around the transition in TPR.

Output

```
experiments/E4_clean_detection/detection_results.parquet
analysis/tpr_vs_length.pdf
analysis/clean_score_distributions.pdf
```

Continue

Target TPR ≥ 0.95 by ≤ 5 kb at FPR $\leq 10^{-3}$.

Day 5 - E5/E6/E7 edit characterization

Question

Which DNA modifications are actually difficult?

Work

- substitutions 0.1%-5%;
- insertions 0.1%-5%;
- deletions 0.1%-5%;
- mixed edits;
- crops 75/50/25/10%;

- reverse complement;
- log induced token errors.

Output

```
experiments/E5_substitutions/edit_results.parquet
experiments/E6_indels/edit_results.parquet
experiments/E7_crop_phase_orientation/results.parquet
analysis/substitution_curve.pdf
analysis/indel_curve_s0.pdf
analysis/crop_phase.pdf
```

Interpretation

If substitutions/crops are manageable but indels collapse detection, that validates synchronization as the next problem rather than invalidating the whole sampling idea.

Day 6 - E8 local resynchronization

Question

Can simple standalone methods confine indel damage?

Implement/compare

- S0 global phase;
- S1 local windows/blocks;
- S2 one simple local-context state variant if time permits;
- S3 lightweight edit-aware alignment prototype.

Output

```
experiments/E8_resynchronization/results.parquet
analysis/error_vs_distance_from_edit.pdf
analysis/sync_method_comparison.pdf
```

Continue

If at least one method restores useful detection under the intended indel range without model replay.

Day 7 - E9/E10/E12 and decision

Work

- wrong-key sweep;
- first system-level null calibration;
- simple unkeyed distinguishers;
- implement/port at least ITS or EXP as the most relevant generic keyed baseline if not already used;
- direct comparison table.

Master table

Method	Clean TPR @ FPR 10^{-3}	minimum clean length	1% substi- tutions	1% indels	50% crop	wrong- key FP	unkeyed AUC	distribution claim
Partition	TBD	TBD	TBD	TBD	TBD	TBD	TBD	exact marginal target
ITS	TBD	TBD	TBD	TBD	TBD	TBD	TBD	[R6]
EXP	TBD	TBD	TBD	TBD	TBD	TBD	TBD	[R6]
other base- line	TBD	TBD	TBD	TBD	TBD	TBD	TBD	verify

Final one-week decision

Choose one:

- **CONTINUE;**
- **CONTINUE BUT REDESIGN SYNCHRONIZATION;**
- **USE GENERIC WATERMARK DIRECTLY;**
- **STOP.**

Do not choose “continue” merely because code was successfully written.

32. Full experiment matrix after the feasibility gate

Only run this grid after the one-week decision supports it.

32.1 Models

- current GENERator-v2 1.2B development checkpoint;
- 1.2B checkpoint from the other domain if relevant;
- 3B confirmation checkpoint;
- original GENERator only if a historical comparison answers a real question.

32.2 Baselines

- B_{tok} primary;
- B_{bp} control.

32.3 Sampling configurations

Use E2 to choose a small realistic set spanning different entropy levels rather than a huge arbitrary grid.

32.4 Lengths

At minimum:

120, 240, 480, 960, 1920, 3840, 7680, 15360 bp.

Add longer genome-scale fragments only if needed for the actual use case.

32.5 Watermark schemes

- Partition P;
- ITS;
- EXP;
- best distribution-preserving baseline;
- practical ECC/coded variant;
- selected PRC if E13 reaches implementation.

32.6 Synchronization

- S0;
- S1 best block/window size;
- S2 if useful;
- S3 if useful;
- S4 only after demonstrated need.

32.7 Edits

- substitutions;
- insertions;
- deletions;
- mixed edits;
- crop;
- reverse complement;
- circular rotation where relevant;
- bounded contiguous window replacement/local resampling for defensive removal testing.

32.8 Primary metrics

Watermark/detection

- TPR at fixed system FPR;
- detection length;
- score separation;
- wrong-key rate;
- fragment sensitivity;
- synchronization recovery length;
- runtime.

Channel

- entropy/token;
- effective vocabulary;
- information/token;
- information/base;
- intrinsic mismatch;
- induced observation substitutions/indels.

Distribution

- exact-marginal unit-test error;
- token/nucleotide frequency shifts;
- unkeyed classifier AUC;
- independent likelihood.

Biology proxies

- GC;
 - nucleotide k -mers;
 - complexity;
 - repeats;
 - ORF summary where appropriate;
 - generic annotation consistency where appropriate.
-

33. Biological and sequence-quality evaluation

The project is about provenance, not biological optimization. Quality evaluation should determine whether the sampler introduces obvious sequence artifacts while avoiding claims that computational plausibility implies function.

Test	What it can tell us	What it cannot establish
GC fraction	large composition shifts	biological function or viability
nucleotide k -mers	local sequence-statistic shifts	phenotype
6-mer token frequencies	tokenizer-level artifacts	fitness
run lengths / complexity	repetitive or degenerate generation artifacts	expression
independent model likelihood	plausibility under another model	experimental validity
ORF summaries in appropriate benign contexts	gross changes in coding structure	protein function
annotation consistency	predicted architecture differences	wet-lab phenotype

33.1 Matched comparisons

For each prompt/configuration compare:

1. B_{tok} ordinary output;
2. watermarked B_{tok} output;
3. B_{bp} output as a separate control.

Do not interpret differences between B_{tok} and B_{bp} as watermark effects.

33.2 Independent scoring

If an independent genomic model is used, freeze it before seeing watermark labels. Report its name/version and exactly what quantity is scored.

Do not score watermarked output only with the same GENERator model whose probabilities drove the sampler and call that independent validation.

33.3 Safe benchmark scope

Use public benign genomic contexts and released model benchmarks. No experiment in this provenance study needs to optimize pathogenic traits, host range, virulence, immune evasion, or any other harmful biological function.

34. Security evaluation by attacker class

34.1 A0 - ordinary edits/no malicious attacker

Examples:

- random substitutions;
- sequencing-like insertions/deletions;
- cropping;
- orientation change.

Purpose: characterize persistence of evidence under ordinary sequence changes.

34.2 A1 - watermark existence known, algorithm details unknown

This is a weak attacker and should not be the main security claim. It is useful only as a baseline.

34.3 A2 - algorithm public, key secret

This should be the default practical security setting.

Attacker may know:

```
GENERATOR checkpoint family
B_tok definition
sampler construction
block size
phase/orientation detector policy
public code
```

but not:

```
K_master
K_partition
K_code
fresh generation-specific secret state
```

Tests:

- generic random edits;
- token-aware but key-unaware edits;
- frequency-based guesses about likely partition structure;
- multi-sample statistical analysis.

34.4 A3 - detector-query access

Example score-query sequence:

```
original DNA          score 0.93
change one candidate region  score 0.90
change another region  score 0.84
...
```

Measure query count and edit budget required to cross threshold.

Binary-query and numerical-score APIs should be separate experiments.

34.5 A4 - many watermarked samples

Attacker receives many generations under the same provider key.

Test for:

- repeated code stream;
- repeated partition schedule;
- token-frequency correlations across samples;
- ability to infer key-dependent groups statistically;
- nonce reuse;
- provider fingerprint leakage beyond intended detection.

The generation protocol must use fresh randomness as required by the selected construction. Reusing the same deterministic target sequence across large numbers of outputs is an avoidable design error.

34.6 A5 - model and decoding policy known

Assume the attacker can reproduce GENERator B_{tok} probabilities for a sequence prefix. This may allow them to identify low/high-entropy positions or search for plausible alternatives.

The key remains secret.

Evaluate bounded generic local resampling as a watermark-removal attack. Do not optimize any harmful biological function.

34.7 A6 - key compromise

If the verification/generation secret is stolen, the attacker can potentially forge or remove evidence depending on the scheme.

State this plainly. The watermark is not a key-management system.

Mitigations such as key rotation, provider epochs, or public-key verification are future protocol work and should not be mixed into the MVP.

35. Proof-obligation ledger

Every major claim should map to an obligation. The table below is the project's claim ledger.

ID	Obligation	Plain-English meaning	Evidence needed	Earliest resolution	Fatal if false?
O1	B_{tok} is precisely defined and reproduced	We know what generator distribution is being preserved	source audit + regression trace	E1	yes
O2	sampler with required random source preserves B_{tok}	watermark does not alter the declared per-step distribution	derivation + E0/E3	E0/E3	yes for exact-distortion claim

ID	Obligation	Plain-English meaning	Evidence needed	Earliest resolution	Fatal if false?
O3	real GENERator has enough stochastic token capacity	4096 nominal tokens translate to usable	E2	E2	yes for current approach
O4	DNA+key detector works without model replay	uncertainty provenance can be checked from sequence alone	implementation + E4	E4	yes for standalone claim
O5	clean finite-length detection is useful	required fragment is not impractically long	E4	E4	yes for practical use
O6	substitutions map to a correctable observation channel	nucleotide substitutions do not immediately erase evidence	E5 + code tests	E5/E13	for substitution claim
O7	crop/strand uncertainty can be calibrated	phase/orientation search does not create uncontrolled FP	E7/E9	E7/E9	yes for fragment claim
O8	standalone indel resynchronization exists	nucleotide indels do not permanently destroy token alignment	E6/E8	E8	yes for indel claim
O9	system-level FPR is controlled	search over phases/windows/keys does not inflate false alarms beyond reported rate	null theory + E9	E9	yes
O10	wrong keys/many samples do not create hidden failure	secret-key behavior is stable under realistic reuse	E9 + A4	E9/security	yes for multi-sample claim
O11	output lacks an easy unkeyed distinguisher	watermark is not trivially visible from token statistics	E10	E10	yes for empirical stealth claim
O12	adaptive removal requires stated edit budget	removal claim is measured rather than assumed	E11	E11	only if claimed

ID	Obligation	Plain-English meaning	Evidence needed	Earliest resolution	Fatal if false?
O13	baseline comparisons are fair	custom method is not winning through different decoding/length/FPR code	E12	E12	yes for comparisons
O14	ECC/PRC assumptions match measured channel	guarantees are not misapplied from token/bit edits to DNA edits	E13	E13	yes for formal edit claim
O15	PRC/sampler composition reduction is valid	cryptographic indistinguishability transfers through autoregressive sampler	proof	after E13	yes for crypto claim

35.1 Plain-English expansion of the most important obligations

O1 - What is the baseline?

If the team cannot say whether the target is the released base-marginal generator or direct token sampling, every later “distribution-preserving” statement is ambiguous.

O2 - Is exact preservation actually true?

For Construction P this is a mathematical/software correctness question. With uniform latent bits, integrating over the hidden bit and then sampling proportionally inside the selected group should reconstruct the original categorical token distribution. E0 must verify the implementation.

For ITS/EXP, use the exact property and assumptions from [R6].

O3 - Does GENERator leave enough randomness?

A 4096-token vocabulary is irrelevant if almost all probability sits on one token. E2 is therefore a project-killing gate.

O4 - Can we verify without GENERator?

If the verifier needs old logits, old prompts, or the original inference environment, the system becomes model-replay provenance rather than the intended sequence-level watermark.

O8 - Can indels resynchronize?

This is the main 6-mer-specific risk. Clean cropping is only six phases; an indel can change the phase of the entire suffix. E8 must show whether local blocks/alignment can recover.

O9 - Is the reported FPR a system FPR?

A low p-value on one chosen window is not meaningful if the implementation tried millions of windows/phases/offsets before selecting it.

O15 - Is the cryptographic composition proof real?

“Exact marginal sampler + PRC” suggests a reduction but does not constitute one. The adaptive autoregressive dependence must be included explicitly.

35.2 Claim-type labels

Every result in the eventual manuscript should be classified as one of:

Mathematical

Example:

$$P_{\text{WM}}(Z_t = z \mid Z_{<t}) = P_{B_{\text{tok}}}(Z_t = z \mid Z_{<t}).$$

Cryptographic

Example:

Under assumption X and construction Y, no polynomial-time observer without the secret can distinguish the two generation processes with advantage exceeding ...

Statistical/empirical

Example:

AUC = 0.51 [95% CI ...]

Biological proxy

Example:

GC and tested k-mer summaries show no measurable shift under this benchmark.

Never convert a statistical result into a cryptographic claim or a biological proxy into a function/viability claim.

36. Research thresholds and why they exist

These thresholds are internal decision aids. They are not constants from watermarking literature.

Quantity	Initial threshold	Why use it?	What happens if missed?
simple unkeyed AUC	≤ 0.55	catches obvious empirical artifacts	inspect sampler/key schedule; narrow stealth claim

Quantity	Initial threshold	Why use it?	What happens if missed?
clean TPR at early FPR	≥ 0.95 by ≤ 5 kb	makes fragment verification practically plausible	reconsider sampler/use case
early FPR	$\leq 10^{-3}$	directly measurable during prototype	lower later; do not deploy at this rate
substitution result	TPR ≥ 0.90 after 1% subs at 5 kb	tests whether local token changes are correctable	improve code/detector
indel result after sync	TPR ≥ 0.85 under 1% total indel at 5 kb	tests whether phase recovery is viable	redesign synchronization/narrow claim
removal attack	warning if <1% generic edits reliably remove	indicates easy adaptive erasure	security redesign/narrow claim
E2 information warning	mean $I_{bp} < 0.01$ and no ≤ 5 kb simulated detection	nominal vocabulary not translating into signal	change decoding/method/stop

The most important threshold is not any single number. It is the combined evidence that a **real GENERator decoding policy** provides enough signal for a **standalone** detector at a sequence length appropriate to the intended use.

37. Twelve-week execution plan

Proceed only if the one-week sprint supports it.

Weeks 1-2 - freeze generator and samplers

Engineering

- productionize B_{tok} and B_{bp} adapters;
- complete Construction P, ITS, EXP;
- key/nonce hierarchy;
- trace logging;
- deterministic configs;
- regression tests.

Theory

- formalize Construction-P exact marginal proof;
- derive/verify random partition variance diagnostic;
- document ITS/EXP properties exactly from [R6];
- define detector-independent notion of sequence-level baseline.

Deliverables

```
docs/baseline_definition.md
docs/sampler_math.md
experiments/E0... complete
experiments/E1... complete
```

Decision

If direct token sampling is a poor model of realistic GENERator use, decide whether the project should instead watermark B_{bp} or explicitly study a modified token-sampling deployment.

Weeks 3-4 - large channel benchmark

Scale E2.

Targets

- millions of token distributions if compute permits;
- at least two context families;
- development checkpoint + confirmation checkpoint subset;
- realistic decoding configurations.

Deliverables

```
data/e2/generator_channel.parquet
figures/channel_entropy.pdf
figures/effective_vocab.pdf
figures/partition_balance.pdf
figures/information_per_bp.pdf
```

Decision

Lock the sampler(s) that move to full evaluation. Drop methods that are clearly dominated.

Weeks 5-6 - standalone detector and finite-length behavior

Engineering

- detector for every retained sampler;
- phase/orientation enumeration;
- local window scanner;
- empirical calibration;
- practical keyed/ECC stream if needed.

Experiments

- E4 clean detection;
- E7 clean crop/orientation;
- first E9 wrong-key calibration.

Deliverables

- clean TPR-vs-length figure;
 - phase/orientation figure;
 - first null table.
-

Weeks 7-8 - nucleotide edits and synchronization

Experiments

- E5 substitutions;
- E6 insertions/deletions;
- mixed edits;
- E8 S0-S3 comparison;
- block-size sweep.

Deliverables

```
figures/substitution_curve.pdf
figures/indel_curve.pdf
figures/error_distance_from_edit.pdf
figures/synchronization_comparison.pdf
```

Decision

If simple local recovery works, do not add S4. If it fails, determine whether explicit synchronization coding is worth the added complexity.

Week 9 - nulls and security evaluation

Run:

- large E9 null corpus;
- wrong-key sweeps;
- A4 many-sample analysis;
- E10 unkeyed classifiers;
- E11 bounded detector-query removal.

Deliverable:

```
security/security_evaluation.md
```

The project should be allowed to lose claims in this week.

Week 10 - baseline replication

Complete E12 with:

- ITS;
- EXP;
- direct protein-WM adaptation if feasible;
- DiPmark adaptation if meaningful;
- simple keyed green/red reference;
- custom partition sampler.

Deliverable: one fair comparison table with matched FPR, length, prompts, and edits.

If a generic method already solves the problem, use it and make synchronization/evaluation the engineering focus.

Week 11 - ECC/PRC and biological/system checks

Coding

- fit practical ECC to measured channel;
- evaluate selected PRC construction/parameters;
- draft composition theorem or state why it remains conditional.

Quality/system

- full distribution checks;
 - independent sequence scoring;
 - GC/k-mers/complexity;
 - runtime/memory;
 - end-to-end reproducibility.
-

Week 12 - decision and paper/technical report skeleton

Write only claims supported by completed obligations.

Possible result categories:

Result A - direct keyed sampler works

Large token support provides short clean detection; substitutions/crops are manageable; local synchronization handles useful indel rates.

Result B - generic text sampler works, custom partition unnecessary

Use EXP/ITS directly and document the 6-mer genomic edit interface.

Result C - clean watermark works but indels defeat standalone verification

Report the boundary honestly and either develop synchronization as a separate project or limit the use case to unedited/cropped/substitution-only DNA.

Result D - direct token distribution is too concentrated

Do not force a watermark onto the model. Reconsider decoding policy or stop.

38. Compute plan

38.1 Benchmark first, do not guess throughput

The released GENERator-v2 path includes custom generation logic. Direct B_{tok} sampling may have different throughput. Benchmark on the actual lab/HPC GPU on Day 1.

Record:

- checkpoint;
- GPU;
- dtype;
- prompt length;

- generated token/base length;
- KV-cache settings;
- batch size;
- tokens/s;
- bases/s;
- peak memory.

38.2 GPU-expensive tasks

Use GPU for:

- collecting real next-token distributions;
- baseline/watermarked generations;
- likelihood calculations requiring large models;
- confirmation checkpoint experiments.

38.3 CPU-cheap tasks

Use CPU for:

- E0 synthetic sampler testing;
- keyed partition generation;
- edit simulation;
- six-phase tokenization;
- detector scans after DNA is generated;
- large wrong-key/null experiments when model replay is unnecessary;
- ECC/PRC channel simulation.

Do not spend GPU time estimating a null tail once the detector is model-independent.

38.4 Storage planning

A million positions with a full 4096-float vector would be large. Prefer storing:

- summary metrics for all positions;
- selected-token data;
- top- N probabilities/token IDs;
- full vectors only for a stratified subset or compressed array store.

If full vectors are needed, use chunked/compressed formats rather than one giant JSONL.

38.5 Confirmation-model strategy

Do not repeat the full 1M-position grid on every 3B checkpoint immediately. First identify the few configurations that determine the conclusion, then confirm those on the larger model.

39. Main risks and stopping conditions

39.1 The real token distribution may still be low entropy

Consequence: the apparent 4096-way advantage is mostly nominal.

Resolve with E2.

39.2 Direct token sampling may be an artificial deployment baseline

Current GENERator-v2 released generation uses B_{bp} , not direct categorical B_{tok} . A reviewer/user may reasonably ask whether modifying decoding to B_{tok} changes generation quality or intended model behavior. Therefore B_{bp} must remain a control and the project must explicitly justify using B_{tok} .

39.3 Generic watermarking may already solve the clean token problem

This is acceptable. Use the generic method directly if it performs best.

39.4 Nucleotide indels may make standalone 6-mer verification impractical

This is the largest structural risk after E2. A single indel changes phase for a suffix. If local block/alignment methods cannot recover at useful edit rates, the project should narrow the persistence claim or invest specifically in synchronization.

39.5 False positives may explode under scanning

Six phases, two orientations, many windows and multiple keys create many hypotheses. System-level calibration is mandatory.

39.6 Key reuse may create repeated patterns

Fresh generation-specific randomness and domain-separated keys must be designed from the start.

39.7 A score-query attacker may erase evidence cheaply

If very few edits remove the watermark, do not describe it as resistant to adaptive removal.

39.8 PRC theory may not be practical at finite genomic lengths

The project can still use a practical detector/ECC and make a narrower statistical provenance claim. Do not pretend a theorem is implemented when it is not.

40. Decision framework

At the end of the feasibility sprint and again after 12 weeks, choose one of four outcomes.

CONTINUE

Choose this when:

1. B_{tok} is reproducible and justified;
2. real distributions show usable stochastic signal;
3. clean DNA+key detection works at useful length;
4. exact/declared distribution tests pass;
5. substitutions/crops behave as expected;
6. at least one standalone synchronization method gives useful indel performance;
7. null/wrong-key behavior can be calibrated.

CONTINUE BUT REDESIGN SYNCHRONIZATION

Choose this when:

- clean detection is good;
- substitutions/crops are good;
- the sampling watermark itself is valid;
- but nucleotide indels cause unacceptable phase collapse.

Then the research focus becomes 6-mer watermark synchronization rather than sampler capacity.

USE GENERIC WATERMARK DIRECTLY

Choose this when ITS, EXP, DiPmark, or the private-key biological-sequence baseline transfers directly and matches or exceeds the custom sampler.

Do not maintain a custom construction without a technical reason.

STOP

Choose this when one or more core requirements fail with no realistic redesign:

- token distribution effectively deterministic under intended decoding;
 - clean standalone detection requires impractically long DNA;
 - exact distribution-preservation implementation cannot be made correct;
 - verifier necessarily requires original model logits/prompt;
 - even very small ordinary edits erase evidence and synchronization is not recoverable;
 - false-positive calibration is incompatible with the intended scanning scale.
-

41. Immediate implementation order

The next actions should be performed in exactly this order unless E1 reveals a different model behavior.

1. **Freeze current GENERator-v2 model/tokenizer/Transformers revisions.**
2. **Write a 20-line tokenizer audit proving which 4096 IDs are canonical DNA 6-mers and which 32 are special.**
3. **Instrument the released `_BPLogitsProcessor`.**
4. **Implement and regression-test B_{tok} separately from B_{bp} .**
5. **Run E0 on synthetic distributions.**
6. **Port EXP and ITS from the original source/paper rather than re-deriving them from memory.**
7. **Collect the first 100k real B_{tok} probability vectors.**
8. **Plot entropy, effective support, q , and information/base.**
9. **Decide whether the channel is worth continuing before writing a PRC implementation.**
10. **If yes, build clean DNA+key phase/orientation detector.**
11. **Only then test substitutions/crops.**
12. **Then measure indel phase collapse.**
13. **Only after observing that failure, choose synchronization machinery.**
14. **Run generic sampler baselines before optimizing the custom partition method.**
15. **Integrate ECC/PRC last.**

This ordering minimizes the risk of spending weeks on coding theory before knowing whether the actual GENERator token distribution and standalone detector support the basic use case.

42. Concise technical verdict

A fixed 6-mer genomic language model is a substantially more natural target for secret-key sampling watermarks than a four-symbol nucleotide sampler **only if its deployed next-token distribution actually uses that larger support**.

For GENERator-v2, the source audit reveals an additional complication: the current released `generate()` path converts the 6-mer probabilities into six position-wise four-base marginal decisions. Therefore a token-level watermark study must explicitly define and evaluate a direct 6-mer sampling baseline B_{tok} , while retaining the released B_{bp} behavior as a control.

If E2 shows a broad enough B_{tok} distribution, generic keyed samplers such as ITS/EXP are plausible immediate candidates. The principal unresolved genomic issue then becomes not clean-channel capacity but the fact that nucleotide insertions/deletions destroy the modulo-six token phase under naïve verification.

The project is therefore technically worth testing, but the evidence should be collected in this order:

baseline correctness → real token entropy → clean standalone detection → DNA edit channel → synchronization

The main negative result to be prepared for is also clear:

The 6-mer vocabulary may solve clean watermark capacity while nucleotide indels make model-free sequence verification impractical without a dedicated synchronization mechanism.

That outcome should be treated as a valid research result rather than hidden by evaluating only substitutions or clean sequences.

43. References

The following references are the primary sources that should be cited and re-checked when implementing or writing claims. URLs are provided for reproducibility.

[R1] GENERator. Wu et al. *GENERator: A Long-Context Generative Genomic Foundation Model*. arXiv:2502.07272.

<https://arxiv.org/abs/2502.07272>

[R2] GENERator-v2. Current GENERator-v2 technical/preprint material and released model family. The exact current report/revision should be frozen in the experiment manifest.

<https://github.com/GenerTeam/GENERator>

[R3] GENERator-v2 1.2B eukaryote model repository and custom source. Includes configuration, tokenizer, and `modeling_generator.py`; the source audit in this document is based on the current official model repository and must be pinned to a revision for experiments.

<https://huggingface.co/GenerTeam/GENERator-v2-eukaryote-1.2b-base>

[R4] GENERator official repository.

<https://github.com/GenerTeam/GENERator>

[R5] Hugging Face Transformers generation code/documentation. Use the exact pinned version matching E1 when auditing processor/warper order.

<https://github.com/huggingface/transformers>

[R6] Kuditipudi et al. *Robust Distortion-free Watermarks for Language Models*. TMLR 2024 / arXiv:2307.15593. Introduces distortion-free keyed sampling constructions including inverse-transform and exponential-minimum approaches and model-agnostic detection/alignment methodology.

<https://arxiv.org/abs/2307.15593>

- [R7] Wu et al.** *A Resilient and Accessible Distribution-Preserving Watermark for Large Language Models (DiPmark)*. ICML 2024, PMLR 235.
<https://proceedings.mlr.press/v235/wu24h.html>
- [R8] Chen et al.** *Enhancing privacy in biosecurity with watermarked protein design*. Bioinformatics 41(7), btaf141, 2025. Private-key watermarking for autoregressive biological protein sequence generation and sequence+key verification.
<https://doi.org/10.1093/bioinformatics/btaf141>
- [R9] Christ, Gunn & Zamir.** *Undetectable Watermarks for Language Models*. COLT 2024, PMLR 247.
<https://proceedings.mlr.press/v247/christ24a.html>
- [R10] Christ & Gunn.** *Pseudorandom Error-Correcting Codes*. CRYPTO 2024 / arXiv:2402.09370.
<https://arxiv.org/abs/2402.09370>
- [R11] Christ, Golowich, Gunn, Moitra & Wicks.** *Improved Pseudorandom Codes from Permuted Puzzles*. STOC 2026 / arXiv:2512.08918.
<https://arxiv.org/abs/2512.08918>
- [R12] Huang et al.** *High-Rate Public-Key Pseudorandom Codes for Edit Errors*. arXiv:2605.19402, 2026.
<https://arxiv.org/abs/2605.19402>
- [R13] Zhang et al.** *Securing the Language of Life: Inheritable Watermarks from DNA Language Models to Proteins*. NeurIPS 2025 / arXiv:2509.18207. DNAMark/CentralMark.
<https://arxiv.org/abs/2509.18207>
- [R14] Golowich & Moitra.** *Edit Distance Robust Watermarks via Indexing Pseudorandom Codes*. NeurIPS 2024.
https://proceedings.neurips.cc/paper_files/paper/2024/hash/24c53bfa5b53fc2cf05644f5a7a26bb0-Abstract-Conference.html
- [R15] Haeupler & Shahrashbi.** *Synchronization Strings: Codes for Insertions and Deletions Approaching the Singleton Bound*. STOC 2017 / arXiv:1704.00807, together with later survey work.
<https://arxiv.org/abs/1704.00807>

Appendix A. Construction-P marginal derivation

For completeness, suppose $C \sim \text{Bernoulli}(1/2)$ and the desired emitted group has marginal

$$Y \sim \text{Bernoulli}(q).$$

For $q \geq 1/2$, define

$$P(Y = 1 \mid C = 1) = 1$$

and

$$P(Y = 1 \mid C = 0) = 2q - 1.$$

Then

$$P(Y = 1) = \frac{1}{2} \cdot 1 + \frac{1}{2}(2q - 1) = q.$$

The mismatch probability is

$$P(Y \neq C) = P(C = 0, Y = 1) = \frac{1}{2}(2q - 1) = q - \frac{1}{2}.$$

For $q < 1/2$, define

$$P(Y = 1 \mid C = 0) = 0$$

and

$$P(Y = 1 \mid C = 1) = 2q.$$

Then

$$P(Y = 1) = \frac{1}{2}(2q) = q$$

and

$$P(Y \neq C) = P(C = 1, Y = 0) = \frac{1}{2}(1 - 2q) = \frac{1}{2} - q.$$

Combining both cases:

$$P(Y \neq C) = |q - 1/2|.$$

After Y is selected, sample token z inside the chosen group with probability

$$P(Z = z \mid Y = g(z)) = \frac{P_t(z)}{P_t(Y = g(z))}.$$

Therefore, when the latent bit is uniform and the coupling gives the required group marginal, the final token marginal reconstructs $P_t(z)$.

Appendix B. Random keyed partition balance heuristic

Let every token independently receive a random binary group label $B_z \sim \text{Bernoulli}(1/2)$ as an analytical approximation to a keyed balanced partition. Define

$$q = \sum_z P(z) B_z.$$

Then

$$E[q] = \frac{1}{2} \sum_z P(z) = \frac{1}{2}.$$

Because the B_z are independent in this approximation,

$$\text{Var}(q) = \sum_z P(z)^2 \text{Var}(B_z) = \frac{1}{4} \sum_z P(z)^2.$$

Define the inverse-Simpson effective vocabulary

$$N_{\text{eff}} = \frac{1}{\sum_z P(z)^2}.$$

Then

$$\text{SD}(q) = \frac{1}{2\sqrt{N_{\text{eff}}}}.$$

Under a rough normal approximation,

$$E|q - 1/2| \approx \frac{1}{\sqrt{2\pi N_{\text{eff}}}}.$$

Illustrative values:

N_{eff}	approximate $E q - 1/2 $
4096	0.0062
1024	0.0125
256	0.0249
64	0.0499
16	0.0997
4	0.1995

This is a useful intuition for E2, not a measured property of GENERator and not an exact formula for a perfectly cardinality-balanced keyed permutation partition.

Appendix C. Detector pseudocode

```
def detect_sequence(dna, verification_key, scheme, config):
    dna = normalize_dna(dna)
    candidates = []

    for orientation, seq in [
        ("forward", dna),
        ("reverse_complement", reverse_complement(dna)),
    ]:
        for phase in range(6):
            tokens = tokenize_aligned_dna(seq, phase=phase)

            for window in enumerate_windows(tokens, config):
                for code_offset in enumerate_code_offsets(window, config):
                    score = scheme.score(
                        window.tokens,
                        verification_key=verification_key,
```

```

        code_offset=code_offset,
        context=window.context,
    )
    candidates.append({
        "score": score,
        "orientation": orientation,
        "phase": phase,
        "window": window,
        "code_offset": code_offset,
    })

best = choose_best_candidate(candidates, scheme)

# IMPORTANT: calibrator was fit on the maximum/best statistic produced
# by the exact same complete search, not on a single chosen window.
global_p = config.calibrator.global_p_value(best["score"])

return DetectionResult(
    score=best["score"],
    global_p_value=global_p,
    decision=(global_p <= config.alpha),
    best_orientation=best["orientation"],
    best_phase=best["phase"],
    best_window_start_bp=best["window"].start_bp,
    best_window_end_bp=best["window"].end_bp,
    best_code_offset=best["code_offset"],
    n_hypotheses=len(candidates),
    n_tokens_used=len(best["window"].tokens),
    diagnostics={"candidates": candidates if config.debug else None},
)

```

For ITS/EXP, replace `scheme.score` with the exact key-alignment statistic from the construction. The global-search calibration principle remains.

Appendix D. Experiment completion checklist

Before E0

- ☐ Key hierarchy specified.
- ☐ Baseline vocabulary fixed.
- ☐ Exact sampler equations independently reviewed.

Before E1

- ☐ Model revision pinned.
- ☐ Tokenizer revision pinned.
- ☐ Transformers/PyTorch versions pinned.

Before E2

- ☐ B_tok regression trace passes.
- ☐ B_bp regression trace passes.
- ☐ E0 marginal tests pass.

- ☐ Storage schema frozen.

Before E3

- ☐ At least 100k real token distributions measured.
- ☐ Realistic decoding settings identified.
- ☐ Channel signal is sufficient to justify generation experiment.

Before E4

- ☐ Distribution-preservation checks pass.
- ☐ Detector has no model/logit dependency.
- ☐ Null families defined before threshold fitting.

Before indel claims

- ☐ Substitution-only behavior characterized.
- ☐ Crop/phase/orientation behavior characterized.
- ☐ S0 failure measured.
- ☐ S1-S3 compared on identical edit realizations.

Before cryptographic claims

- ☐ Actual PRC/construction selected.
- ☐ Security assumptions copied accurately from primary source.
- ☐ Fresh randomness/key reuse policy specified.
- ☐ Composition proof drafted and independently reviewed.
- ☐ DNA edit channel mapped to code channel.

Before final report/paper

- ☐ System-level FPR, not only per-window FPR, reported.
- ☐ Baselines use matched prompts/decoding/lengths.
- ☐ Unkeyed distinguisher uses held-out keys/prompts.
- ☐ Biological proxies are not described as viability/function proof.
- ☐ Negative results and stopping conditions are included.
- ☐ B_tok vs released B_bp distinction appears in abstract/methods if relevant.